

Gestion de projets

Plan :

Partie I: gérer un projet de A à Z

Chapitre 1 : Introduction à la gestion de projet informatique

- Notion de projet
- Spécificités des projets informatiques
- Contraintes fondamentales d'un projet
- Typologies de projets informatiques
- Enjeux de la gestion de projet informatique
- Rôle de la gestion de projet informatique

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

- Introduction
- Les parties prenantes d'un projet informatique
- La Maîtrise d'Ouvrage (MOA)
- La Maîtrise d'Œuvre (MOE)
- L'Assistance à Maîtrise d'Ouvrage (AMOA)
- Le chef de projet informatique
- L'équipe projet
- Gouvernance des projets IT
- Communication et coordination entre acteurs
- Facteurs clés de succès liés aux acteurs

Chapitre 3 : Les étapes projet

- L'avant-projet.
- La conception et la réalisation
- La validation
- La recette et la réception
- La livraison
- Le bilan de fin de projet

Chapitre 4 : Les cycles de vie

- Les normes
- Les modèles séquentiels
- Le cycle de vie « itératif »
- Les méthodes « agiles »
- Les bonnes pratiques

Chapitre 5 : Le suivi du projet

- Les réunions
- La communication
- Les indicateurs et les tableaux de bord
- La gestion du portefeuille projet

Chapitre 6 : Organiser le changement

- L'accompagnement au changement
- La conduite du changement

Chapitre 1 : Introduction à la gestion de projet informatique

- 1.1. Notion de projet
- 1.2. Spécificités des projets informatiques
- 1.3. Contraintes fondamentales d'un projet
- 1.4. Typologies de projets informatiques
- 1.5. Enjeux de la gestion de projet informatique
- 1.6. Rôle de la gestion de projet informatique

Chapitre 1: Introduction à la gestion de projet informatique

1.1. Notion de projet

1.1.1. Définition d'un projet

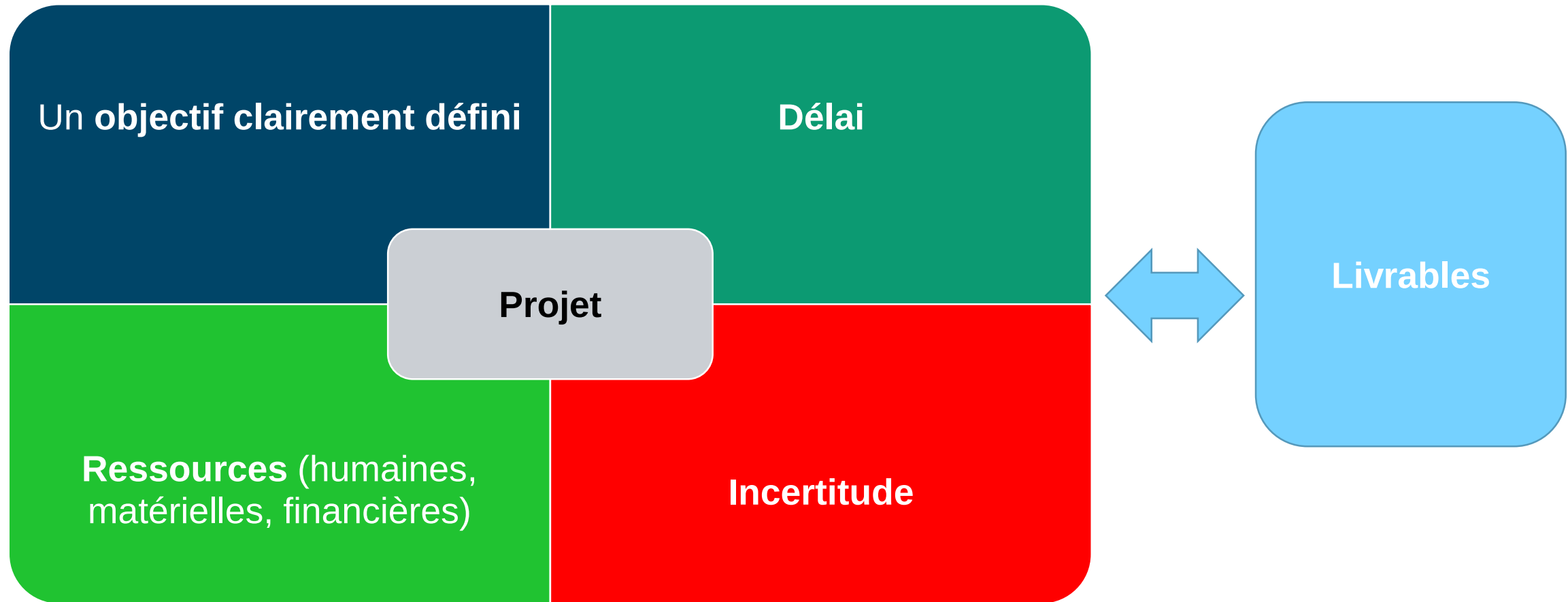
Un **projet** est un ensemble d'activités **temporaires**, **organisées** et **coordonnées**, visant à atteindre un **objectif précis**, par la production d'un **résultat unique** (produit, service ou solution), dans le respect de contraintes prédéfinies.

Un projet se caractérise donc par :

- une **date de début** et une **date de fin**,
- un **objectif clairement défini**,
- un **livrable identifiable**,
- des **ressources limitées** (humaines, matérielles, financières),
- un **niveau d'incertitude** plus ou moins élevé.

Chapitre 1: Introduction à la gestion de projet informatique

1.1. Notion de projet



Chapitre 1: Introduction à la gestion de projet informatique

1.1. Notion de projet

1.1.2. Différence entre projet et activité opérationnelle

Projet	Activité opérationnelle
Temporaire	Permanente
Objectif unique	Processus répétitif
Forte incertitude	Faible incertitude
Innovation fréquente	Routine

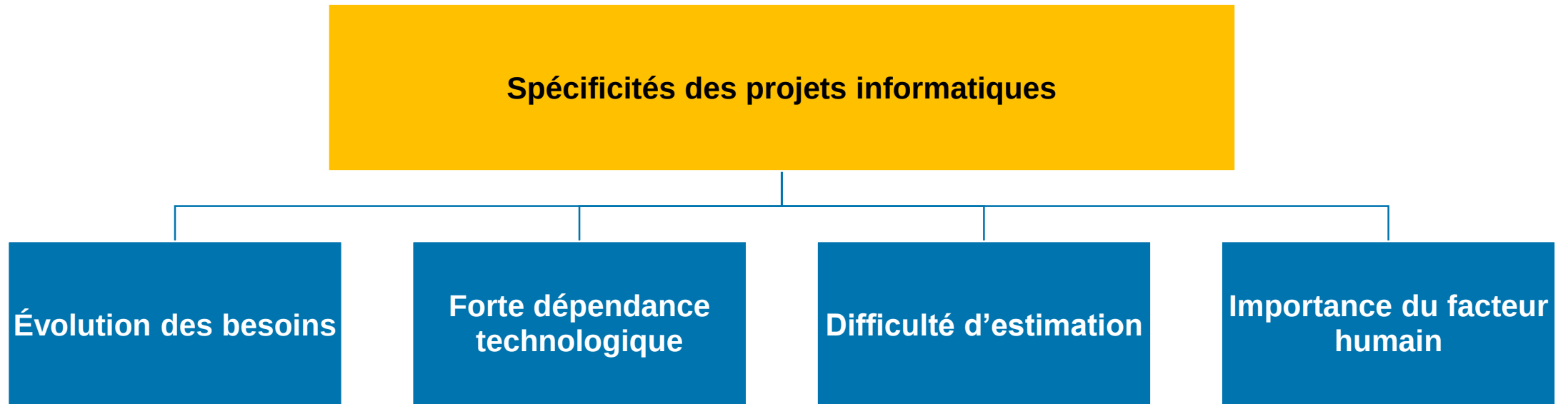
1.1.3. Programme et portefeuille de projets

- **Programme** : ensemble de projets liés entre eux, coordonnés pour atteindre un objectif stratégique commun.
- **Portefeuille de projets** : ensemble de projets et programmes pilotés au niveau stratégique d'une organisation.

Chapitre 1: Introduction à la gestion de projet informatique

1.2. Spécificités des projets informatiques

Les projets informatiques présentent des **particularités** qui les rendent plus complexes que d'autres types de projets.



Chapitre 1: Introduction à la gestion de projet informatique

1.2. Spécificités des projets informatiques

1.2.1. Évolution des besoins

- Les besoins des utilisateurs sont souvent mal définis au départ
- Ils évoluent avec le temps et l'usage
- Risque élevé de changements en cours de projet

1.2.2. Forte dépendance technologique

- Technologies rapidement obsolètes
- Dépendance aux outils, frameworks et plateformes
- Problèmes de compatibilité et d'intégration

1.2.3. Difficulté d'estimation

- Estimation imprécise des charges et des délais
- Dépendance aux compétences de l'équipe
- Incertitude sur les risques techniques

1.2.4. Importance du facteur humain

- Travail collaboratif
- Communication essentielle
- Changements possibles dans l'équipe

Chapitre 1: Introduction à la gestion de projet informatique

1.3. Contraintes fondamentales d'un projet

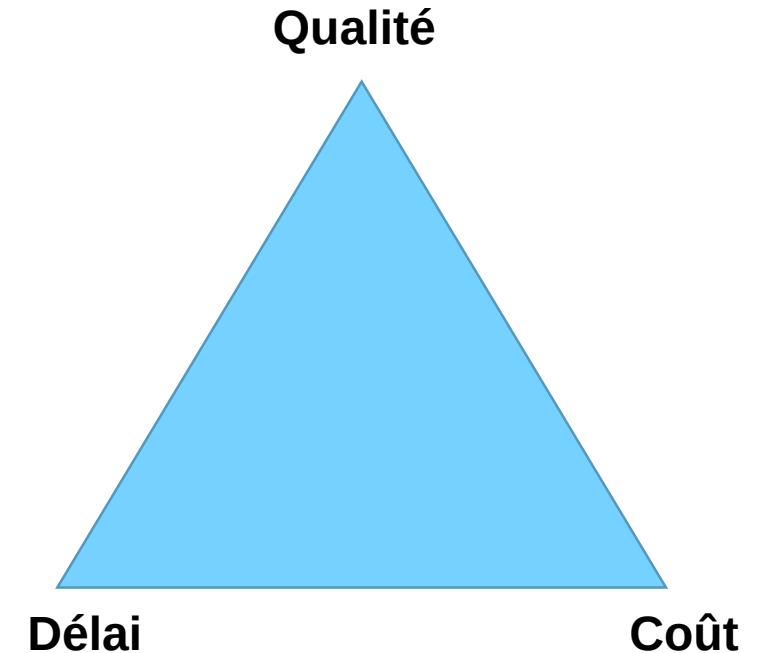
1.3.1. Le triangle de gestion de projet (QCD)

Tout projet est soumis à trois contraintes majeures :

- **Qualité** : conformité aux besoins et exigences
- **Coût** : respect du budget alloué
- **Délai** : respect des échéances

Ces contraintes sont **interdépendantes** :

→ Modifier l'une impacte nécessairement les autres.



Chapitre 1: Introduction à la gestion de projet informatique

1.3. Contraintes fondamentales d'un projet

1.3.2. Le périmètre du projet

Le **périmètre** définit :

- ce qui est **inclus** dans le projet,
- ce qui est **exclu**.

Une mauvaise définition du périmètre entraîne :

- dérive des besoins,
- dépassement des délais et des coûts.

Chapitre 1: Introduction à la gestion de projet informatique

1.4. Typologies de projets informatiques

1.4.1. Projets logiciels

- Applications web
- Applications mobiles
- Logiciels métiers

1.4.2. Projets d'infrastructure IT

- Réseaux informatiques
- Serveurs
- Data centers
- Sécurité informatique

1.4.3. Projets data et intelligence artificielle

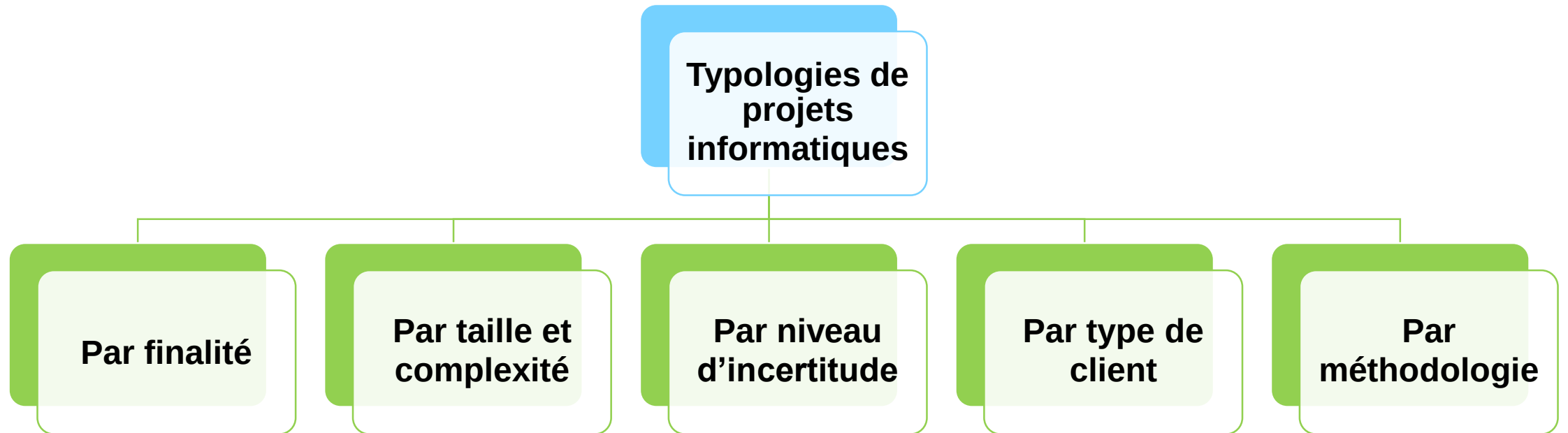
- Big Data
- Data warehouses
- Machine Learning
- Business Intelligence

1.4.4. Projets de transformation digitale

- Dématérialisation des processus
- E-gouvernement
- Digitalisation des services

Chapitre 1: Introduction à la gestion de projet informatique

1.4. Typologies de projets informatiques



Chapitre 1: Introduction à la gestion de projet informatique

1.4. Typologies de projets informatiques

❑ Par finalité

- **Développement applicatif** : nouvelles applis, ERP, mobile, outils métier.
- **Infrastructure** : serveurs, cloud, virtualisation, datacenter.
- **Refonte/modernisation** : migration legacy, amélioration maintenabilité.
- **Sécurité** : RGPD, cybersécurité, durcissement SI.

❑ Par taille et complexité

- **Petits projets** : <6 mois, <5 pers., cycles courts, organisation légère.
- **Moyens projets** : 6 mois–2 ans, 5–20 pers., organisation structurée, suivi formel.
- **Grands projets** : >2 ans, >20 pers., sous-projets, gouvernance complexe.
- **Programmes** : regroupent plusieurs projets interdépendants.

Chapitre 1: Introduction à la gestion de projet informatique

1.4. Typologies de projets informatiques

❑ Par type de client

- Interne : au sein de l'organisation, relations directes.
- Externe : client extérieur, relations contractuelles.
- Mixte/partenarial : plusieurs organisations, gouvernance partagée.

❑ Par niveau d'incertitude

- Routine : besoins stables, faible risque.
- Amélioration : innovation modérée, risque moyen.
- Innovation : expérimentations nouvelles, risque élevé.
- Rupture : transformation radicale, très haut risque.

❑ Par méthodologie

- Cascade (Waterfall) : séquentiel, validation formelle, livraison unique.
- Agile (Scrum, Kanban) : itérations courtes, livraisons fréquentes, adaptation continue.
- Hybride : mix Cascade + Agile, prédictibilité + flexibilité.

Chapitre 1: Introduction à la gestion de projet informatique

1.5. Enjeux de la gestion de projet informatique

La gestion de projet informatique regroupe l'ensemble des méthodes, outils et compétences permettant de **planifier, organiser, piloter et contrôler** des projets de **systèmes d'information** et de développement logiciel. Elle vise à livrer **un produit ou un service conforme aux besoins du client**, dans le respect des contraintes de **coût, délais, qualité et risques**.

Chapitre 1: Introduction à la gestion de projet informatique

1.5. Enjeux de la gestion de projet informatique

Enjeux

Stratégiques

Économiques

Techniques

Humains &
organisationnels

Gouvernance & pilotage

Chapitre 1: Introduction à la gestion de projet informatique

1.5. Enjeux de la gestion de projet informatique

❑ Stratégiques

- Transformation digitale : digitalisation, nouveaux modèles, expérience client.
- Avantage concurrentiel : innovation rapide, optimisation coûts...
- Conformité & risques : RGPD, normes, sécurité.
- Agilité organisationnelle : systèmes évolutifs, adaptabilité.

❑ Économiques

- Maîtrise des coûts et respect du budget.
- Retour sur investissement (ROI) : gains quantifiables et qualitatifs.
- Coût du retard : opportunités perdues, pénalités.
- Optimisation du portefeuille de projets.

Chapitre 1: Introduction à la gestion de projet informatique

1.5. Enjeux de la gestion de projet informatique

❑ Techniques

- Maîtrise de la complexité et architecture modulaire.
- Qualité et robustesse : réduction de la dette technique.
- Intégration et interopérabilité.
- Sécurité : Security by Design.

❑ Humains & organisationnels

- Gestion du changement et adoption utilisateur.
- Mobilisation et montée en compétences des équipes.
- Collaboration transverse entre métiers et IT.
- Gestion du stress et motivation.

❑ Gouvernance & pilotage

- Visibilité et transparence pour sponsors et direction.
- Prise de décision éclairée (technique, périmètre, ressources).
- Alignement stratégique et ajustement en continu.
- Capitalisation et amélioration continue.

Chapitre 1: Introduction à la gestion de projet informatique

1.5. Enjeux de la gestion de projet informatique

Une bonne gestion de projet informatique permet de :

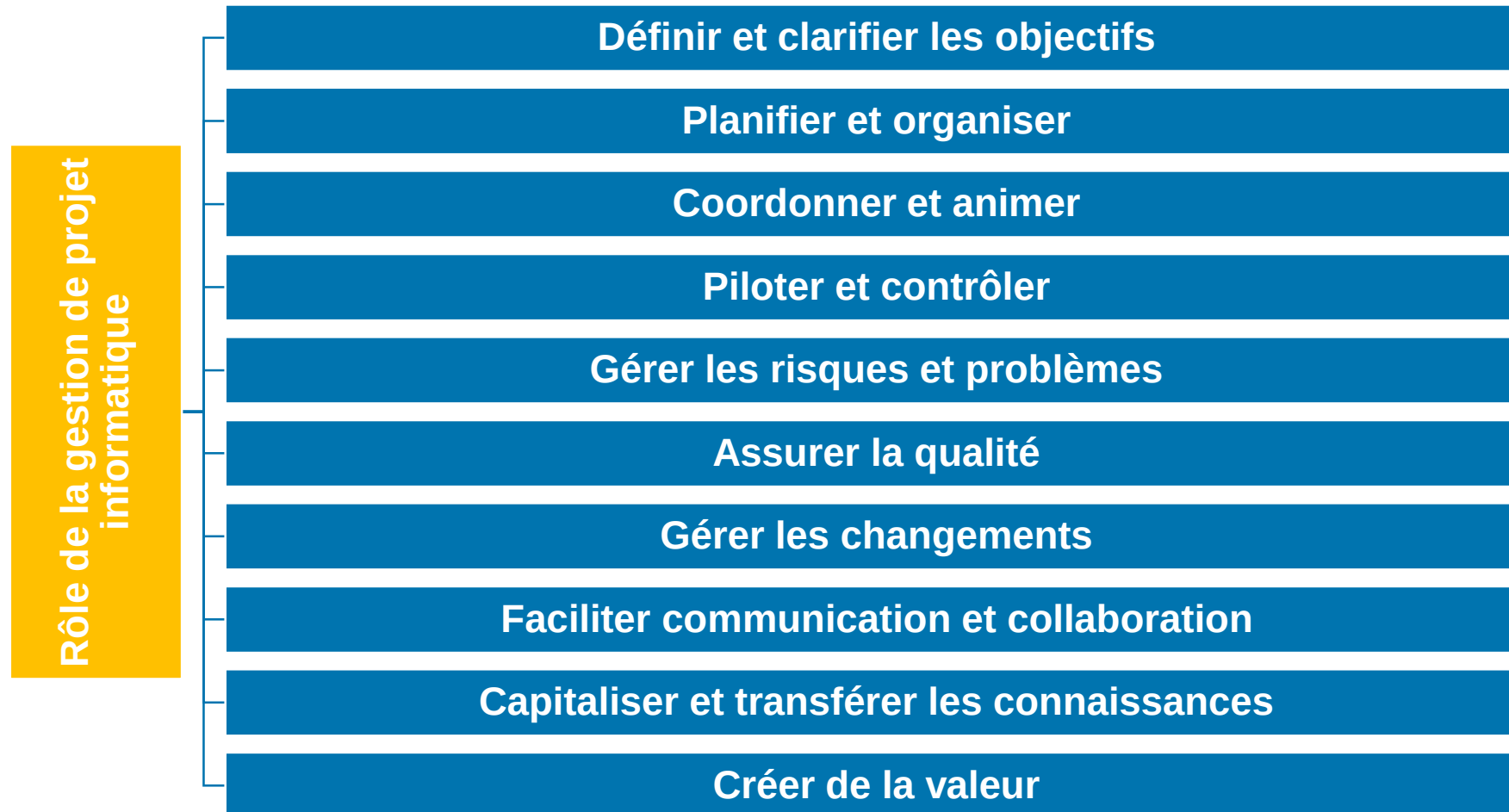
- ✓ maîtriser les coûts,
- ✓ respecter les délais,
- ✓ améliorer la qualité des livrables,
- ✓ réduire les risques,
- ✓ satisfaire les utilisateurs.

À l'inverse, une mauvaise gestion peut entraîner :

- échec du projet,
- dépassement budgétaire,
- rejet par les utilisateurs,
- pertes financières importantes.

Chapitre 1: Introduction à la gestion de projet informatique

1.6. Rôle de la gestion de projet informatique



Chapitre 1: Introduction à la gestion de projet informatique

1.6. Rôle de la gestion de projet informatique

❑ Définir et clarifier les objectifs

- Traduire les besoins métier en spécifications techniques.
- Délimiter le périmètre et fixer les critères de succès.
- Aligner toutes les parties prenantes.

❑ Planifier et organiser

- Découper le projet en tâches gérables.
- Estimer ressources et budget, séquencer les tâches.
- Définir rôles, responsabilités et allocation des ressources.

❑ Coordonner et animer

- Dynamiser l'équipe et maintenir la motivation.
- Faciliter la communication et la collaboration.
- Assurer la cohérence entre équipes et intervenants.

Chapitre 1: Introduction à la gestion de projet informatique

1.6. Rôle de la gestion de projet informatique

❑ **Piloter et contrôler**

- Suivre l'avancement, le budget et la qualité.
- Détecter les écarts et prendre des mesures correctives.
- Reporting transparent et gestion de configuration.

❑ **Gérer les risques et problèmes**

- Identifier, évaluer et prioriser les risques.
- Planifier réponses et gérer crises ou problèmes.

❑ **Assurer la qualité**

- Définir standards, mettre en place processus et contrôles.
- Suivre l'amélioration continue.

Chapitre 1: Introduction à la gestion de projet informatique

1.6. Rôle de la gestion de projet informatique

❑ Gérer les changements

- Processus formel de modification et adaptation du projet.
- Protection de l'équipe contre les interruptions fréquentes.

❑ Faciliter communication et collaboration

- Communication vers le haut, horizontal et vers le bas.
- Gestion des parties prenantes et engagement adapté.

❑ Capitaliser et transférer les connaissances

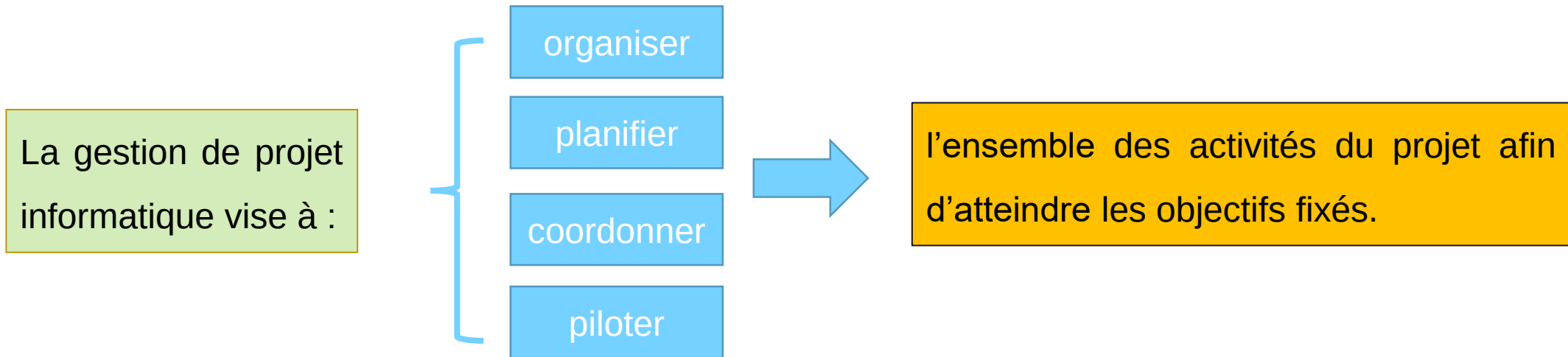
- Documentation, transfert vers exploitation et retour d'expérience.
- Amélioration continue des pratiques.

❑ Créer de la valeur

- Optimiser les ressources et réduire les gaspillages.
- Accélérer le time-to-value et gérer l'incertitude.

Chapitre 1: Introduction à la gestion de projet informatique

1.6. Rôle de la gestion de projet informatique



Elle constitue aujourd'hui une **compétence clé** pour les ingénieurs et les responsables IT.

Le projet informatique est une **activité complexe**, marquée par l'incertitude, le changement et la dépendance technologique.

La **gestion de projet informatique** est donc indispensable pour garantir le succès des projets IT.

Chapitre 1: Introduction à la gestion de projet informatique

1.6. Rôle de la gestion de projet informatique

❑ Trois types de gestion:

- Gestion de la **productivité**: organisation, direction de travaux et contrôle (suivi)
- Gestion des **ressources humains et matériels**: choix de l'équipe, affectation de personnel, coordination, sélection matériel, gestion du budget et contrôle des coûts
- Gestion du **temps** pour la maîtrise des délais

❑ Ces trois types de gestion sont interdépendants

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

- 2.1. Introduction
- 2.2. Les parties prenantes d'un projet informatique
- 2.3. La Maîtrise d'Ouvrage (MOA)
- 2.4. La Maîtrise d'Œuvre (MOE)
- 2.5. L'Assistance à Maîtrise d'Ouvrage (AMOA)
- 2.6. Le chef de projet informatique
- 2.7. L'équipe projet
- 2.8. Gouvernance des projets IT
- 2.9. Communication et coordination entre acteurs
- 2.10. Facteurs clés de succès liés aux acteurs

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.1. Introduction

Un projet informatique ne repose pas uniquement sur la technologie. Son succès dépend largement de la **coordination entre les acteurs**, de la **clarté des rôles** et de la **qualité de la gouvernance** mise en place.

De nombreux échecs de projets IT sont dus à :

- ☐ une mauvaise définition des responsabilités,
- ☐ des conflits entre acteurs,
- ☐ une absence de mécanismes de décision clairs,
- ☐ une communication inefficace.

☞ Ce chapitre vise à identifier **tous les acteurs d'un projet IT**, leurs **rôles respectifs** et les **structures de gouvernance** permettant de piloter efficacement le projet.

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.2. Les parties prenantes d'un projet informatique

2.2.1. Définition des parties prenantes

Les **parties prenantes** sont toutes les personnes, groupes ou organisations :

- ☐ impliqués dans le projet,
- ☐ ou pouvant influencer son déroulement,
- ☐ ou étant impactés par ses résultats.

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.2. Les parties prenantes d'un projet informatique

2.2.2. Typologie des parties prenantes

On distingue :

a) Parties prenantes internes

- Direction générale
- Direction des systèmes d'information (DSI)
- Chef de projet
- Équipe projet
- Utilisateurs internes

b) Parties prenantes externes

- Clients
- Fournisseurs
- Prestataires
- Consultants
- Partenaires technologiques

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.2. Les parties prenantes d'un projet informatique

2.2.3. Importance de l'identification des parties prenantes

Une mauvaise identification des parties prenantes peut entraîner :

- des résistances au changement,
- des conflits,
- des retards,
- un rejet du projet.

👉 **La gestion des parties prenantes consiste à :**

- identifier leurs attentes,
- analyser leur pouvoir et leur influence,
- adapter la communication.

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.3. La Maîtrise d'Ouvrage (MOA)

2.3.1. Définition de la MOA

La **Maîtrise d'Ouvrage (MOA)** représente le **client** ou le **donneur d'ordre** du projet. Elle est responsable de l'**expression des besoins** et de la **validation des résultats**.

2.3.2. Rôles et responsabilités de la MOA

- ✓ définit les objectifs du projet,
- ✓ exprime les besoins fonctionnels,
- ✓ rédige ou valide le **cahier des charges**,
- ✓ valide les livrables,
- ✓ finance le projet,
- ✓ décide des orientations majeures.

☞ La MOA répond à la question :
« **Que faut-il faire ?** »

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.4. La Maîtrise d'Œuvre (MOE)

2.4.1. Définition de la MOE

La **Maîtrise d'Œuvre (MOE)** est responsable de la **réalisation technique** du projet.
Elle conçoit, développe et met en œuvre la solution informatique.

2.4.2. Rôles et responsabilités de la MOE

- ✓ analyse la faisabilité technique,
- ✓ conçoit l'architecture du système,
- ✓ développe la solution,
- ✓ réalise les tests,
- ✓ déploie le système,
- ✓ assure la maintenance technique.

☞ La MOE répond à la question :
« **Comment le faire ?** »

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.5. L'Assistance à Maîtrise d'Ouvrage (AMOA)

2.5.1. Définition de l'AMOA

L'**AMOA** est un acteur **intermédiaire** entre la MOA et la MOE.

Elle accompagne la MOA dans la définition, le pilotage et la réussite du projet.

2.5.2. Rôles de l'AMOA

- ✓ aide à l'expression des besoins,
- ✓ formalise les exigences fonctionnelles,
- ✓ participe au choix des solutions,
- ✓ suit l'avancement du projet,
- ✓ facilite la communication MOA–MOE,
- ✓ accompagne la conduite du changement.

2.5.3. Intérêt de l'AMOA

L'AMOA est particulièrement utile lorsque :

- ✓ la MOA manque de compétences techniques,
- ✓ le projet est complexe,
- ✓ plusieurs prestataires sont impliqués.

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

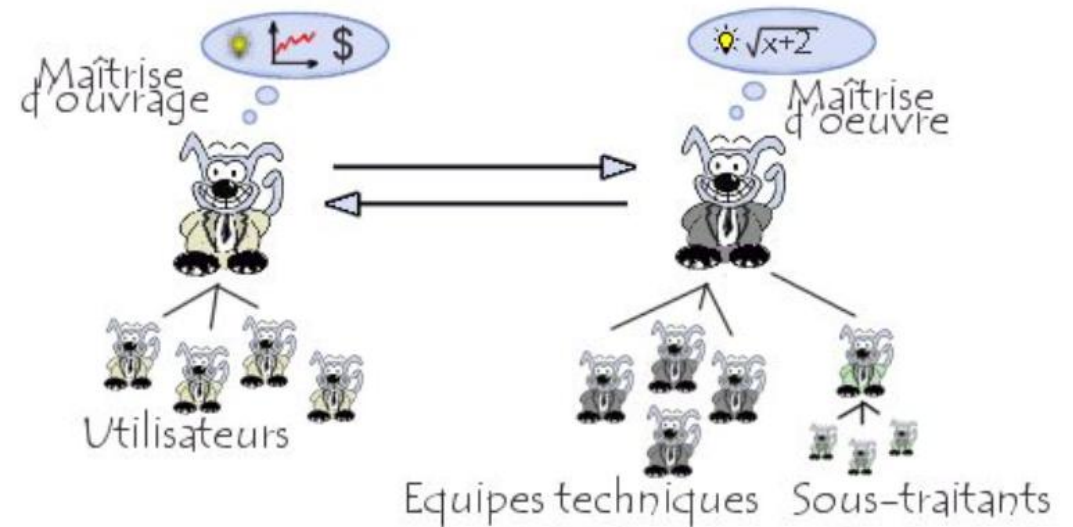
Maître d'œuvre et maître d'ouvrage

Un projet vise à satisfaire un ensemble de **besoins** exprimés par des futurs utilisateurs. Mais il est conduit à terme par des **spécialistes** en développement de SI



Que faut-il faire ?

Comment le faire ?



Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.6. Le chef de projet informatique

2.6.1. Définition

Le **chef de projet informatique** est le responsable opérationnel du projet.

Il coordonne l'ensemble des acteurs afin d'atteindre les objectifs du projet.

2.6.2. Missions principales

- planifie les activités,
- organise les ressources,
- suit l'avancement,
- gère les risques,
- assure la communication,
- rend compte à la gouvernance.

2.6.3. Positionnement du chef de projet

Il se situe :

- entre la MOA et la MOE,
- entre la direction et l'équipe projet.

Il joue un rôle clé de **coordination et de médiation**.

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.7. L'équipe projet

2.7.1. Composition de l'équipe projet

Une équipe projet IT peut inclure :

- développeurs,
- architectes,
- testeurs,
- analystes fonctionnels,
- administrateurs systèmes,
- experts métiers.

2.7.2. Fonctionnement de l'équipe projet

- Travail collaboratif
- Répartition claire des rôles
- Responsabilisation
- Communication régulière

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.8. Gouvernance des projets IT

2.8.1. Définition de la gouvernance

La **gouvernance du projet** désigne l'ensemble des **mécanismes de décision**, de **pilotage** et de **contrôle** permettant d'assurer l'alignement du projet avec la stratégie de l'organisation.

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.8. Gouvernance des projets IT

2.8.2. Les instances de gouvernance

a) Comité de pilotage

- Instance stratégique
- Prend les décisions majeures
- Valide les orientations et les livrables clés

b) Comité projet

- Instance opérationnelle
- Suit l'avancement
- Gère les problèmes quotidiens

2.8.3. Rôle de la gouvernance

La gouvernance permet de :

- clarifier les responsabilités,
- accélérer la prise de décision,
- maîtriser les risques,
- assurer la cohérence stratégique.

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.9. Communication et coordination entre acteurs

2.9.1. Importance de la communication

Une communication défailante est l'une des principales causes d'échec des projets IT.

2.9.2. Outils de communication

- Réunions
- Rapports d'avancement
- Tableaux de bord
- Outils collaboratifs (Teams, Jira, Trello)

Chapitre 2 : Acteurs, rôles et gouvernance des projets IT

2.10. Facteurs clés de succès liés aux acteurs

- ☐ Clarté des rôles
- ☐ Engagement de la MOA
- ☐ Compétence du chef de projet
- ☐ Implication des utilisateurs
- ☐ Gouvernance efficace

Le succès d'un projet informatique repose avant tout sur les **acteurs**, leurs **rôles clairement définis** et une **gouvernance efficace**.

La technologie seule ne suffit pas : la dimension humaine et organisationnelle est déterminante.

Chapitre 3 : Les étapes projet

- L'avant-projet.
- La conception et la réalisation
- La validation
- La recette et la réception
- La livraison
- Le bilan de fin de projet

Chapitre 3 : Les étapes projet

Introduction

- ❑ La **gestion d'un projet informatique** repose sur une démarche structurée composée de plusieurs **phases**, de **la conception à la clôture**. Chaque phase a des objectifs, des livrables et des critères de validation précis.
- ❑ Selon la **méthodologie utilisée**, ces étapes peuvent s'adapter ou se chevaucher, mais l'essentiel est d'assurer une **progression maîtrisée, documentée et validée** par l'ensemble des parties prenantes.

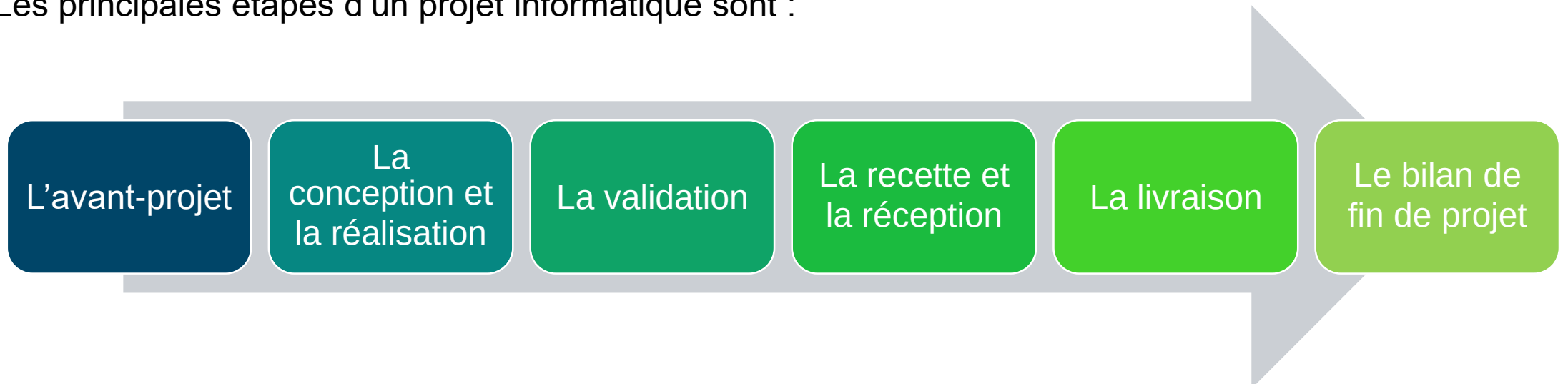
Chapitre 3 : Les étapes projet

Introduction

Chaque étape a des **objectifs précis**, des **acteurs impliqués**, des **activités spécifiques** et des **livrables attendus**. Le respect de ces étapes permet :

- de réduire les risques,
- d'assurer la qualité du produit final,
- de maîtriser les coûts et les délais,
- de garantir la satisfaction du client.

Les principales étapes d'un projet informatique sont :



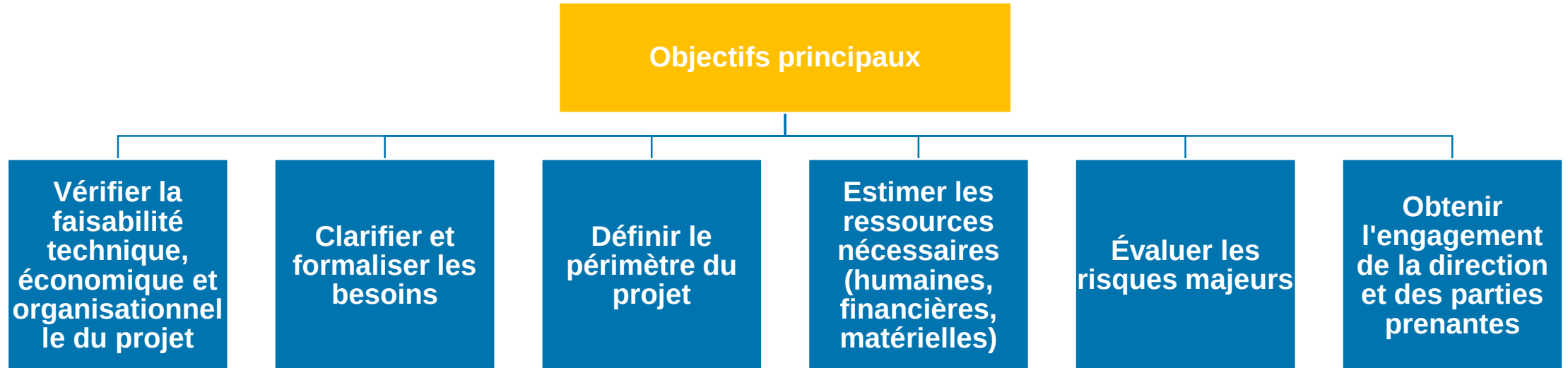
Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.1 Définition et objectifs

L'avant-projet constitue **la phase préparatoire et stratégique du projet**. Elle vise à **transformer une idée ou un besoin exprimé en un projet formalisé et validé** par toutes les parties prenantes. Cette étape est cruciale car elle conditionne la réussite de l'ensemble du projet.



Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.2 Les activités de l'avant-projet

a) L'étude d'opportunité

b) L'étude de faisabilité

c) L'expression des besoins

d) La définition du périmètre

e) L'analyse des risques
préliminaire

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.2 Les activités de l'avant-projet

a) L'étude d'opportunité

Objectif : Décider si le projet doit être lancé.

1. Alignement stratégique

- Cohérence avec la vision et les objectifs de l'organisation
- Contribution à la stratégie globale

2. Analyse de la valeur

- Estimation des bénéfices :
 - Gains de productivité
 - Réduction des coûts
 - Nouveaux revenus
 - Amélioration de l'image
- Comparaison avec les coûts estimés

3. Analyse de l'urgence

- Besoin immédiat ou différable ?

4. Étude des alternatives

- Ne rien faire
- Améliorer l'existant
- Acheter une solution du marché
- Développer en interne

5. Analyse coûts–bénéfices préliminaire

- Coûts : développement, déploiement, formation, maintenance
- Bénéfices : quantitatifs et qualitatifs

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.2 Les activités de l'avant-projet

b) L'étude de faisabilité

Objectif : Vérifier si le projet est réalisable et viable

Faisabilité technique

- Technologies adaptées au besoin
- Disponibilité des compétences (interne, formation, sous-traitance)
- Contraintes majeures : performance, volumétrie, sécurité
- Compatibilité avec l'infrastructure et les systèmes existants

Faisabilité organisationnelle

- Impact sur les processus et les rôles
- Nouveaux besoins en compétences
- Capacité à gérer le changement
- Identification et gestion des résistances

Faisabilité économique

- Estimation détaillée des coûts :
 - Développement (équipe, outils, licences)
 - Déploiement (infrastructure, migration)
 - Formation
 - Maintenance (corrective & évolutive)
- Projection des gains : productivité, réduction des coûts, nouveaux revenus
- Calcul du **ROI** et du **délai de récupération**

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.2 Les activités de l'avant-projet

c) L'expression des besoins

Objectif : Identifier et formaliser les attentes des utilisateurs et parties prenantes.

Recueil des besoins

- Entretiens individuels (utilisateurs clés, experts métier)
- Ateliers collaboratifs
- Observation sur le terrain
- Questionnaires

Hiérarchisation des besoins

- Indispensables
- Importants
- Souhaitables (Nice to have)
- Permet d'arbitrer en cas de contraintes de délais ou de budget

Formalisation des besoins

Besoins fonctionnels

- Fonctionnalités
- Traitements
- Règles de gestion

Besoins non fonctionnels

- Performance (temps de réponse, débit)
- Sécurité (authentification, confidentialité)
- Ergonomie & accessibilité
- Disponibilité
- Maintenabilité

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.2 Les activités de l'avant-projet

d) La définition du périmètre

Objectif : Délimiter clairement le champ du projet.

Éléments inclus

- Fonctionnalités à développer
- Utilisateurs concernés (profils, nombre, localisation)
- Systèmes interfacés
- Limites organisationnelles et géographiques
 - Sites, départements, pays

Éléments exclus

- Fonctionnalités hors projet
- Usages non couverts

Enjeux du périmètre

- Clarifier les attentes
- Éviter les malentendus
- Éviter le dépassements de coûts
- Éviter les retards de planning

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.2 Les activités de l'avant-projet

e) L'analyse des risques préliminaire

Objectif : Identifier et anticiper les menaces pouvant compromettre le projet.

Typologie des risques

Risques techniques

- Complexité élevée
- Technologies nouvelles ou immatures
- Dépendances externes
- Contraintes de performance

Risques organisationnels

- Ressources clés limitées
- Résistance au changement
- Conflits inter-départements
- Instabilité organisationnelle

Risques liés aux parties prenantes

- Faible implication du sponsor ou des utilisateurs
- Attentes irréalistes
- Désaccords sur les priorités

Risques externes

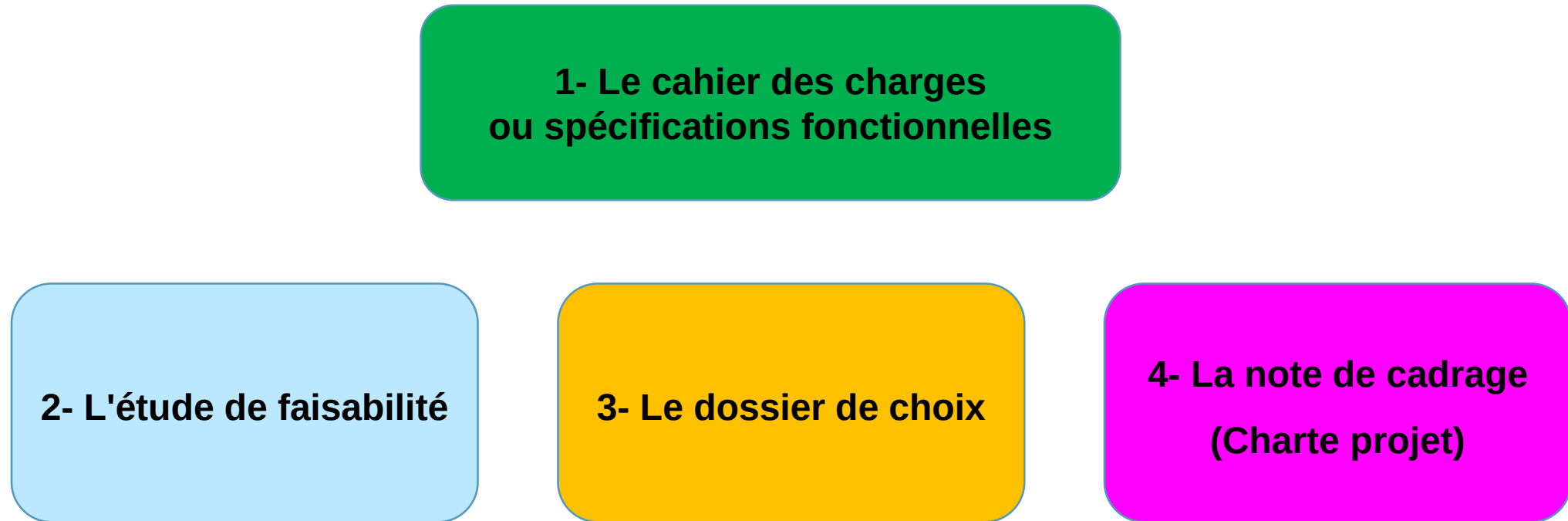
- Évolutions réglementaires
- Dépendance aux fournisseurs
- Contraintes budgétaires ou politiques

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.3 Les livrables de l'avant-projet



Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.3 Les livrables de l'avant-projet

Le cahier des charges ou spécifications fonctionnelles

Document contractuel qui définit les besoins, les objectifs du projet, les contraintes, ainsi que les critères d'acceptation du système à réaliser.

L'étude de faisabilité

Document d'analyse qui évalue les scénarios possibles, leurs coûts, leurs risques, et propose la solution la plus adaptée au projet.

Le dossier de choix

Document de décision qui propose les solutions techniques envisagées, compare les alternatives (développement interne, progiciel, externalisation), justifie le choix retenu, et présente le planning et le budget prévisionnels.

La note de cadrage ou charte projet

Document de cadrage qui définit le cadre du projet : objectifs, périmètre, contraintes, parties prenantes, responsabilités, et organisation du projet.

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.4 Les acteurs impliqués

Maître d'Ouvrage (MOA)

- ❖ Représente les **utilisateurs**
- ❖ Exprime et formalise les **besoins métier**
- ❖ Définit les **objectifs**
- ❖ **Valide** les livrables de l'avant-projet
- ❖ **Finance** le projet
(Direction générale)

Utilisateurs finaux

- ❖ Participent aux **entretiens et ateliers**
- ❖ Apportent la **connaissance métier**
- ❖ **Valident** les besoins et la solution finale

Experts métier

- ❖ Maîtrise approfondie du **domaine fonctionnel**
- ❖ Formalisent les **règles de gestion**
- ❖ Valident la **pertinence fonctionnelle**

Maître d'Œuvre (MOE)

- ❖ Responsable de la **solution technique**
- ❖ Participe à l'**étude de faisabilité**
- ❖ Propose les **solutions techniques**
- ❖ Estime les **charges et coûts**
(DSI ou prestataire externe)

Direction générale

- ❖ Valide l'**opportunité stratégique**
- ❖ Alloue les **budgets**
- ❖ **Arbitre** les priorités et conflits

Experts techniques

- ❖ Architectes, sécurité, DBA, etc.
- ❖ Analysent la **faisabilité technique**
- ❖ Identifient les **contraintes et risques**
- ❖ Proposent des **orientations techniques**

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.5 Validation et décision de lancement

Décisions possibles

❖ Lancer le projet

- Opportunité confirmée
- Faisabilité validée
- Budget alloué

→ Passage en **phase de conception / réalisation**

❖ Différer le projet

- Contexte défavorable (budget, priorités)
- Projet jugé pertinent à moyen terme

❖ Demander des compléments d'étude

- Incertitudes à lever
 - Scénarios à approfondir
- Prolongation de l'avant-projet

❖ Abandonner le projet

- Faisabilité non démontrée
- ROI insuffisant
- Risques trop élevés

Instance de décision

- ❖ **Comité de pilotage** (ou équivalent)
- ❖ Analyse de **l'ensemble des livrables** de l'avant-projet0

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.5 Validation et décision de lancement

Actions en cas de décision positive

- ☐ **Officialisation du lancement** (note de cadrage / lettre de mission)
- ☐ **Nomination du chef de projet**
- ☐ **Allocation du budget**
- ☐ **Engagement des ressources**
- ☐ **Validation du planning macro**
- ☐ **Constitution de l'équipe projet**

Chapitre 3 : Les étapes projet

3.1 L'avant-projet

3.1 L'avant-projet

3.1.6. Durée et importance de l'avant-projet

Durée de l'avant-projet

- ❑ **Petit projet** : quelques jours
 - Projet de courte durée (quelques semaines)
 - ❑ **Projet moyen** : 2 à 6 semaines
 - Projet sur plusieurs mois
 - ❑ **Grand projet** : plusieurs mois
 - Projet complexe, pluriannuel
- ☞ La durée dépend de la **taille**, de la **complexité** et des **enjeux** du projet

Importance stratégique

- ❑ Phase **clé** pour la réussite du projet
- ❑ La phase avant-projet → Réduction possible de **50% des risques d'échec**
- ❑ Évite les erreurs majeures :
 - Besoins mal compris
 - Faisabilité surestimée
 - Risques sous-estimés

Valeur ajoutée de l'avant-projet

- ❑ **Alignement** des parties prenantes
- ❑ **Fondations solides** pour la suite du projet
- ❑ Décision basée sur des **données objectives**

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.1. Vue d'ensemble

La phase de conception et réalisation transforme les besoins exprimés en **solution opérationnelle**. Elle se décompose généralement en deux grandes activités : la **conception** qui définit "comment" le système répondra aux besoins, et la **réalisation** qui construit effectivement le système.

Selon la méthodologie adoptée, ces activités peuvent être séquentielles (d'abord toute la conception, puis toute la réalisation) ou imbriquées (conception et réalisation par itérations successives).

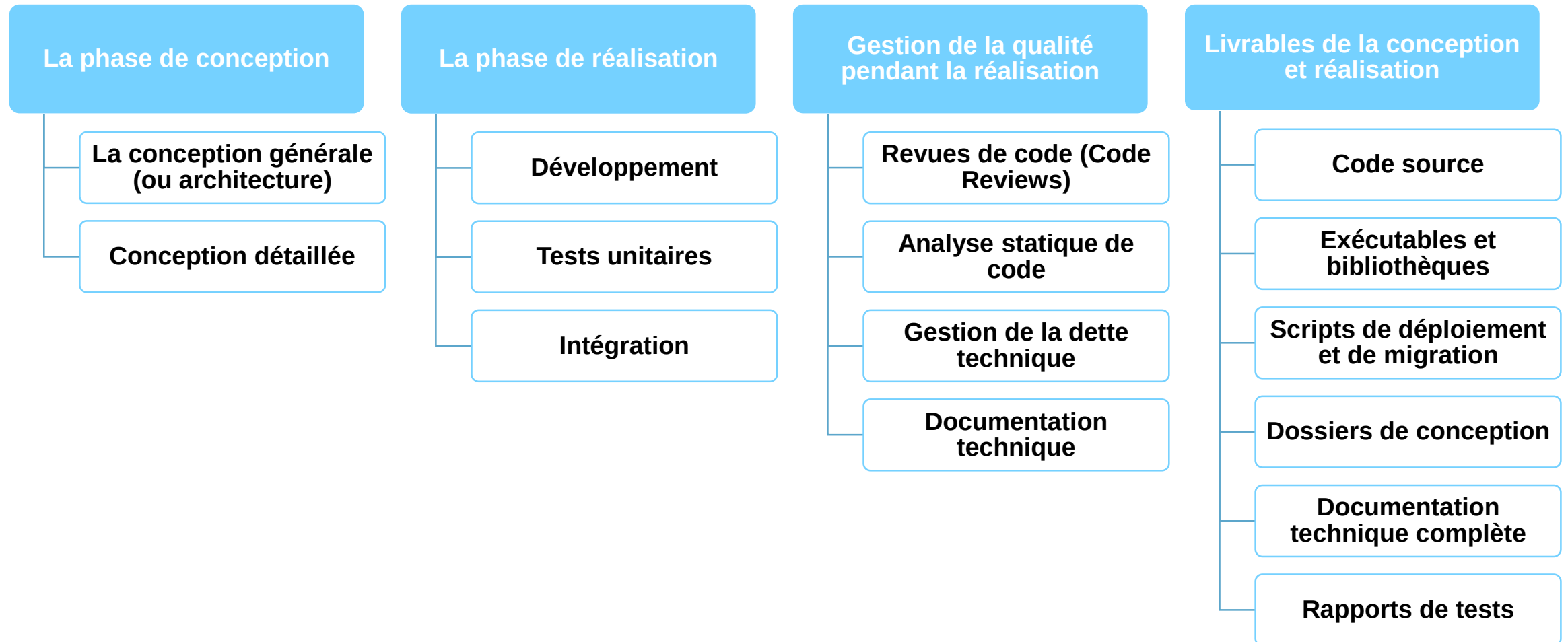
Cette étape correspond au **cœur du projet informatique**.

Elle transforme les besoins exprimés en une **solution informatique concrète**.

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation



Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

Objectif de la conception

- ☐ Traduire les **besoins fonctionnels** → en **spécifications techniques**
- ☐ Préparer une **implémentation maîtrisée**
- ☐ Préparer la phase de réalisation
- ☐ Réduire les risques techniques et fonctionnels

👉 Deux niveaux :

- ☐ **Conception générale**
- ☐ **Conception détaillée**

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

a) La conception générale (ou architecture)

Définition :

C'est la première étape de conception qui définit l'**architecture globale du système**, les grands modules, leurs responsabilités, leurs interactions, l'architecture technique, la sécurité et la disponibilité.

Objectif :

Fournir une **vision globale** du système et orienter les choix techniques et organisationnels, avant de détailler l'implémentation.

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

a) La conception générale (ou architecture)

Exemples de diagrammes utilisés :

- ❖ **Diagramme de composants UML** : montre les modules principaux et leurs interactions
- ❖ **Diagramme de séquence UML (simplifié)** : visualise les échanges de données entre modules pour un scénario principal
- ❖ **Diagramme de couches (layered architecture)** : présente la séparation présentation / métier / persistance / infrastructure
- ❖ **Schéma d'architecture technique / réseau** : topologie réseau, serveurs, flux, zones de sécurité

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

a) Conception générale (Architecture)

Architecture applicative

- Découpage en **modules / composants**
 - Ex. e-commerce : catalogue, panier, commandes, clients, paiement, logistique
- Définition des **interactions entre composants**
 - Échanges de données
 - Protocoles de communication
- **Architecture en couches** :
 - Présentation
 - Métier
 - Persistance
 - Infrastructure
- Modélisation : **diagrammes UML** (composants, séquences)

Architecture technique

- Définition de l'**infrastructure**
- Choix technologiques :
 - Langages & frameworks
 - Bases de données
 - Serveurs & middleware
- **Dimensionnement** :
 - Serveurs, réseau, stockage
 - Cloud vs On-premise
- **Architecture réseau** :
 - Topologie
 - Zones de sécurité (DMZ)
 - Load balancing

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

a) Conception générale (Architecture)

Sécurité (Security by Design)

- **Authentification** : login, SSO, MFA
- **Autorisation** : rôles, permissions
- **Chiffrement** :
 - Données en transit (HTTPS/TLS)
 - Données au repos
- **Traçabilité & logs**
- Mécanismes de protection : Pare-feu, IDS/IPS, Gestion des vulnérabilités

Montée en charge & disponibilité

- **Scalabilité** : Verticale et Horizontale
- Architecture **stateless**
- Mise en cache
- **Haute disponibilité** : Redondance, Failover, Réplication des données

Livrable – Conception générale

- **DCG : Dossier de Conception Générale**
 - Diagrammes d'architecture
 - Choix techniques justifiés
 - Flux & interfaces
 - Principes de sécurité
 - Contraintes de performance

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

b) Conception détaillée

Définition :

C'est l'étape qui **affine chaque composant** identifié en conception générale en spécifiant tous les éléments nécessaires au développement : algorithmes, traitements, structures de données, interfaces, API, règles de gestion et gestion des erreurs.

Objectif :

Fournir aux développeurs toutes les **spécifications précises** permettant l'implémentation correcte du système.

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

b) Conception détaillée

Exemples de diagrammes utilisés :

- ❑ **Diagramme de classes UML** : définit les classes, attributs, méthodes et relations
- ❑ **Diagramme de séquence détaillé**
- ❑ **Modèle conceptuel, logique et physique de données (MCD / MLD / MPD)** : décrit les structures de données et bases de données
- ❑ **Wireframes / maquettes d'interface** : structure des écrans et navigation
- ❑ **Diagramme d'activité UML** : décrit les algorithmes et flux de traitement détaillés

Exemple concret :

Pour le module gestion commandes :

- Classe Commande avec attributs id, client, articles, total et méthodes calculerTotal(), valider()
- Interface utilisateur : écran de validation de commande avec champs et boutons précis

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

b) Conception détaillée

Spécifications des composants

- ❖ Détail des **traitements et algorithmes**
- ❖ Gestion des cas particuliers et erreurs
- ❖ Définition précise des **structures de données**

Interfaces utilisateur (UI/UX)

- ❖ **Wireframes** : structure des écrans
- ❖ **Mockups** : design graphique
- ❖ **Prototypes interactifs**
- ❖ Ergonomie & accessibilité : WCAG, Responsive design

Modélisation des données

- ❖ Bases relationnelles /Bases NoSQL

API et interfaces techniques

- ❖ Spécification des services :
 - Endpoint & méthodes HTTP
 - Paramètres et validations
 - Formats de réponse
 - Gestion des erreurs
- ❖ Protocoles :
 - REST, SOAP
 - Messaging (RabbitMQ, JMS)

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.2 La phase de conception

b) Conception détaillée

Règles de gestion détaillées

- Formalisation **sans ambiguïté**
- Règles conditionnelles claires
- Validations et messages d'erreur

Gestion des erreurs & exceptions

- Erreurs techniques & fonctionnelles
- Timeout & retry
- Mécanismes de fallback
- Journalisation pour le support

Livrable – Conception détaillée

•DCD : Dossier de Conception Détaillée

- Spécifications techniques complètes
- Modèles de données
- Maquettes & ergonomie
- API & protocoles
- Matrice de traçabilité

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.3 La phase de réalisation

Définition :

La phase de réalisation correspond au **développement effectif de la solution**. Elle inclut l'écriture du code source selon des **standards et bonnes pratiques**, la **gestion des versions**, le **développement itératif et incrémental**, les **tests unitaires** et **l'intégration progressive des composants**. Elle intègre également la gestion de la qualité du code via revues, analyse statique, documentation technique et contrôle de la dette technique.

Objectif :

Produire une **solution fonctionnelle, fiable et maintenable**, respectant les spécifications, prête à être testée et intégrée, tout en assurant la qualité et la traçabilité du code.

La phase de réalisation transforme les spécifications en code fonctionnel et testé, tout en assurant la qualité, la traçabilité et la maintenabilité du système.

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.3 La phase de réalisation

a) Le développement

Il s'agit de la réalisation du système en respectant des standards de codage, des bonnes pratiques, une organisation modulaire du code et un suivi par version, avec un développement itératif et des livraisons progressives.

b) Les tests unitaires

Les développeurs vérifient chaque composant de façon isolée afin de s'assurer du bon fonctionnement des fonctionnalités, y compris les cas d'erreur, en privilégiant l'automatisation des tests.

c) L'intégration

Les différents composants sont assemblés progressivement pour tester leurs interactions, garantir la cohérence du système et détecter les incompatibilités, souvent à l'aide d'outils d'intégration continue.

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.3 La phase de réalisation

Objectifs de la réalisation

- ☐ Transformer la **conception** en **code exécutable**
- ☐ Construire la solution informatique
- ☐ Garantir la **qualité**, la **maintenabilité** et la **robustesse**
- ☐ Livrer des incréments **fonctionnels et testés**

Activités de réalisation

- ☐ Développement des composants logiciels
- ☐ Configuration des systèmes
- ☐ Intégration des modules
- ☐ Gestion des versions
- ☐ Documentation technique

Livrables de la réalisation

- ☐ Code source
- ☐ Versions intermédiaires
- ☐ Documentation technique

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.3 La phase de réalisation

a) Organisation du développement

Standards de développement

- ❑ Conventions de **nommage** (variables, classes, tables...)
- ❑ Règles de **structuration du code**
 - Organisation des dossiers
 - Fonctions/méthodes de taille maîtrisée
 - Séparation des responsabilités
- ❑ **Templates et patterns** pour un code homogène

Bonnes pratiques de développement

- ❑ Application des principes **SOLID**
- ❑ Utilisation de **design patterns** :
 - MVC, Factory, Singleton
 - Observer, Strategy
- ❑ **Documentation du code** :
 - Commentaires utiles
 - JavaDoc / Sphinx
 - README par module

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.3 La phase de réalisation

a) Organisation du développement

Gestion de versions

- ☐ Outil de versioning : **Git / SVN**
- ☐ Traçabilité complète des changements
- ☐ Stratégie de branches :
 - **Main / Master** : production
 - **Feature branches** : développements
 - **Release branches** : préparation livraison
 - **Hotfix** : corrections urgentes

Développement itératif & incrémental

- ☐ Développement par **itérations courtes**
 - Sprints (2–4 semaines)
- ☐ Livraison progressive des **fonctionnalités prioritaires**
- ☐ Chaque itération produit un **incrément livrable**
- ☐ Avantages :
 - Feedback rapide
 - Adaptation continue
 - Réduction des risques

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.3 La phase de réalisation

b) Les tests unitaires

Principe

- ☐ Test de chaque **unité de code** isolément
- ☐ Simulation des dépendances (mocks, stubs)

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.3 La phase de réalisation

b) Les tests unitaires

Cas de test

- ☐ **Cas nominaux** (fonctionnement normal)
- ☐ **Cas limites** (valeurs extrêmes)
- ☐ **Cas d'erreur** (données invalides, exceptions)

Automatisation

- ☐ Frameworks :
 - JUnit (Java)
 - pytest (Python)
 - Jest (JavaScript)
 - NUnit (C#), PHPUnit (PHP)
- ☐ Exécution fréquente (idéalement **à chaque commit**)

Couverture de code

- ☐ Mesure :
 - Lignes exécutées
 - Branches testées
 - Fonctions couvertes
- ☐ Objectif courant : **70–80 %**
- ☐ ⚠ Couverture ≠ qualité → importance des assertions

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.3 La phase de réalisation

c) L'intégration

Approches d'intégration

- **Intégration Continue (CI) :**
 - Build automatique
 - Exécution des tests
 - Notification des résultats
- Outils : Jenkins, GitLab CI, GitHub Actions, CircleCI
- Intégration progressive :
 - Bottom-up
 - Top-down
 - Par sous-systèmes

Tests d'intégration

- Vérification du fonctionnement **global**
- Cohérence des échanges entre composants
- Tests avec :
 - Bases de données
 - Services externes
 - Systèmes legacy

Gestion des conflits

- Conflits possibles :
 - Modifications concurrentes
 - Dépendances circulaires
 - Versions incompatibles
- Intégration fréquente :
 - Détection rapide
 - Résolution facilitée

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.4. Gestion de la qualité pendant la réalisation

La **gestion de la qualité pendant la réalisation** vise à garantir que le logiciel développé **respecte les standards, fiable, maintenable et conforme aux spécifications**. Cette étape accompagne le développement en mettant en place des pratiques de contrôle et d'amélioration continue. Elle inclut la **revue du code, l'analyse statique, la maîtrise de la dette technique et la documentation**, afin de prévenir les erreurs, faciliter la maintenance et assurer une livraison de qualité aux utilisateurs finaux.

Objectifs

- Garantir la **qualité du code**
- Réduire les **bugs** et risques de sécurité
- Assurer la **maintenabilité** et l'évolutivité

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.4. Gestion de la qualité pendant la réalisation

Revue de code (Code Reviews)

- ☐ Examen du code par les **pairs**
- ☐ Vérification :
 - Respect des standards
 - Détection de bugs et failles de sécurité
 - Amélioration : performance, lisibilité, maintenabilité
- ☐ **Partage des connaissances**

Formats

- ☐ Formelles : inspections structurées
- ☐ Informelles : pair programming, pull/merge requests

Outils

- ☐ GitHub Pull Requests
- ☐ GitLab Merge Requests
- ☐ Gerrit

Analyse statique de code

- ☐ Analyse automatique **sans exécution**
- ☐ Détection de :
 - Bugs potentiels
 - Failles de sécurité (SQL injection, XSS)
 - Complexité excessive
 - Code mort ou dupliqué
 - Violations des standards

Outils courants

- ☐ SonarQube (multi-langages)
- ☐ ESLint (JavaScript)
- ☐ Pylint (Python)
- ☐ RuboCop (Ruby)
- ☞ Intégration possible dans le **pipeline CI**
 - Blocage du code de mauvaise qualité

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.4. Gestion de la qualité pendant la réalisation

Gestion de la dette technique

❑ Définition :

- Raccourcis techniques à court terme
- Impact négatif à long terme

❑ Exemples :

- Code non optimal
- Absence de tests
- Documentation insuffisante
- Architecture peu évolutive

Conséquences

- ❑ Maintenance plus difficile
- ❑ Augmentation des bugs
- ❑ Ralentissement des évolutions

Gestion

- ❑ Identifier et mesurer (ex. SonarQube)
- ❑ Documenter (backlog)
- ❑ Prioriser (critique vs acceptable)
- ❑ Réduire progressivement (refactoring)
- ⚠ Objectif : **dette maîtrisée**, pas zéro dette

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.4. Gestion de la qualité pendant la réalisation

Documentation technique

- **DAT** : Dossier d'Architecture Technique
- **Guide d'installation** : prérequis & procédures
- **Documentation API** :
 - Swagger / OpenAPI
- **Guides de configuration**
- **Procédures de déploiement**
- ☞ Essentielle pour :
 - Maintenance
 - Onboarding des développeurs
 - Support et exploitation

Chapitre 3 : Les étapes projet

3.2. La Conception et la Réalisation

3.2. La Conception et la Réalisation

3.2.5. Livrables de la conception et réalisation

Les principaux livrables de cette phase sont le **code source** versionné et documenté dans le repository (Git, SVN), les **exécutables et bibliothèques** compilés et packagés, les **scripts de déploiement** et de migration de base de données, les **dossiers de conception** (DCG, DCD) mis à jour si nécessaire, la **documentation technique** complète et à jour, et les **rapports de tests unitaires** et d'intégration avec les taux de couverture.

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.1 Objectifs de la phase de validation

La phase de validation consiste à **vérifier que le système développé répond aux exigences définies dans le cahier des charges**. C'est la dernière étape avant la mise en production, destinée à s'assurer que le système est prêt à être utilisé sans risque majeur.

Objectif :

- ☐ Vérifier la **conformité** du système au **cahier des charges**
- ☐ Confirmer la **conformité du système** aux spécifications fonctionnelles et techniques.
- ☐ Détecter et corriger les **bugs ou anomalies critiques** avant la mise en production.
- ☐ Vérifier les **performances, la sécurité, la compatibilité et l'ergonomie**
- ☐ S'assurer du **bon fonctionnement** dans toutes les situations prévues
- ☐ Tester le comportement face à des **cas imprévus**
- ☐ Vérifier la **conformité aux normes et réglementations**
- ☐ Garantir que la **documentation et les procédures** sont complètes et fiables.
- ☐ Obtenir l'**approbation formelle** des responsables pour passer à la phase de recette et au déploiement.

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.1 Objectifs de la phase de validation

Pourquoi la validation est critique ?

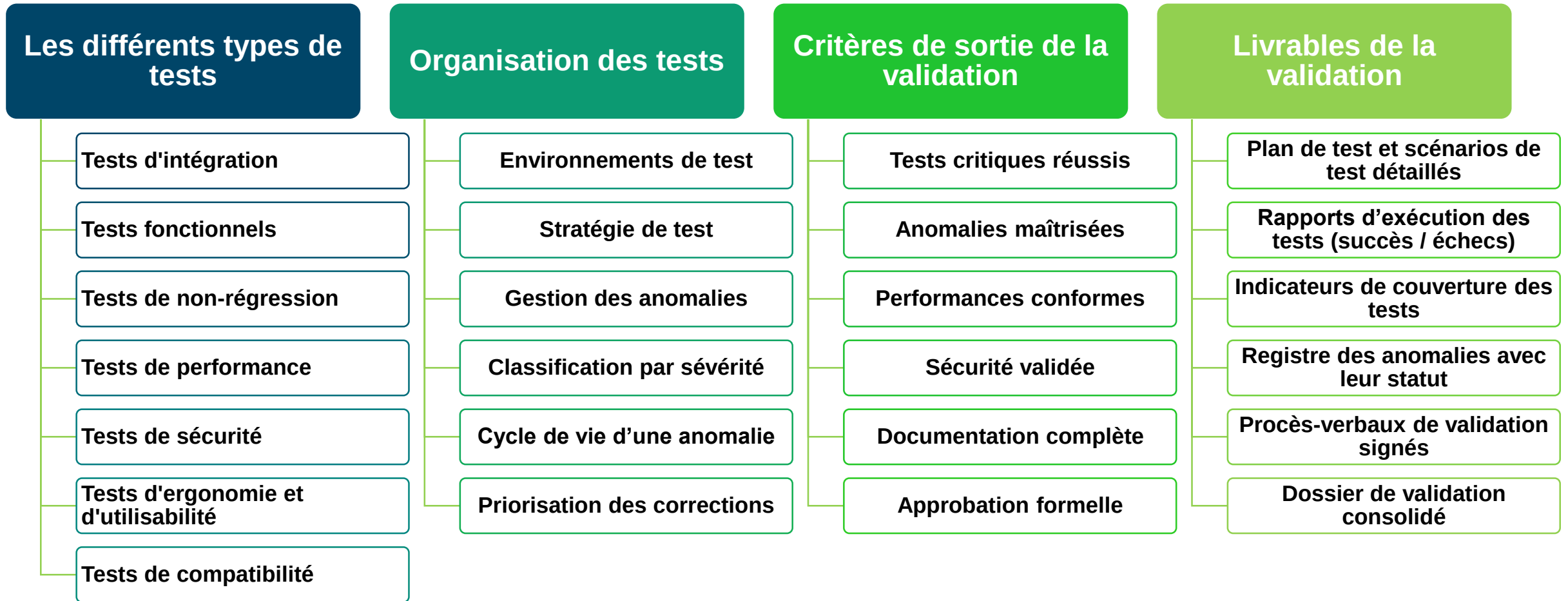
- **Dernière barrière** avant la mise en production
- Les anomalies non détectées :
 - Impactent directement les **utilisateurs**
 - Peuvent générer des **risques métier, financiers ou juridiques**
- Conditionne la **qualité perçue** du système

☞ Un système validé est un système **fiable, conforme et déployable sans risque majeur**

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

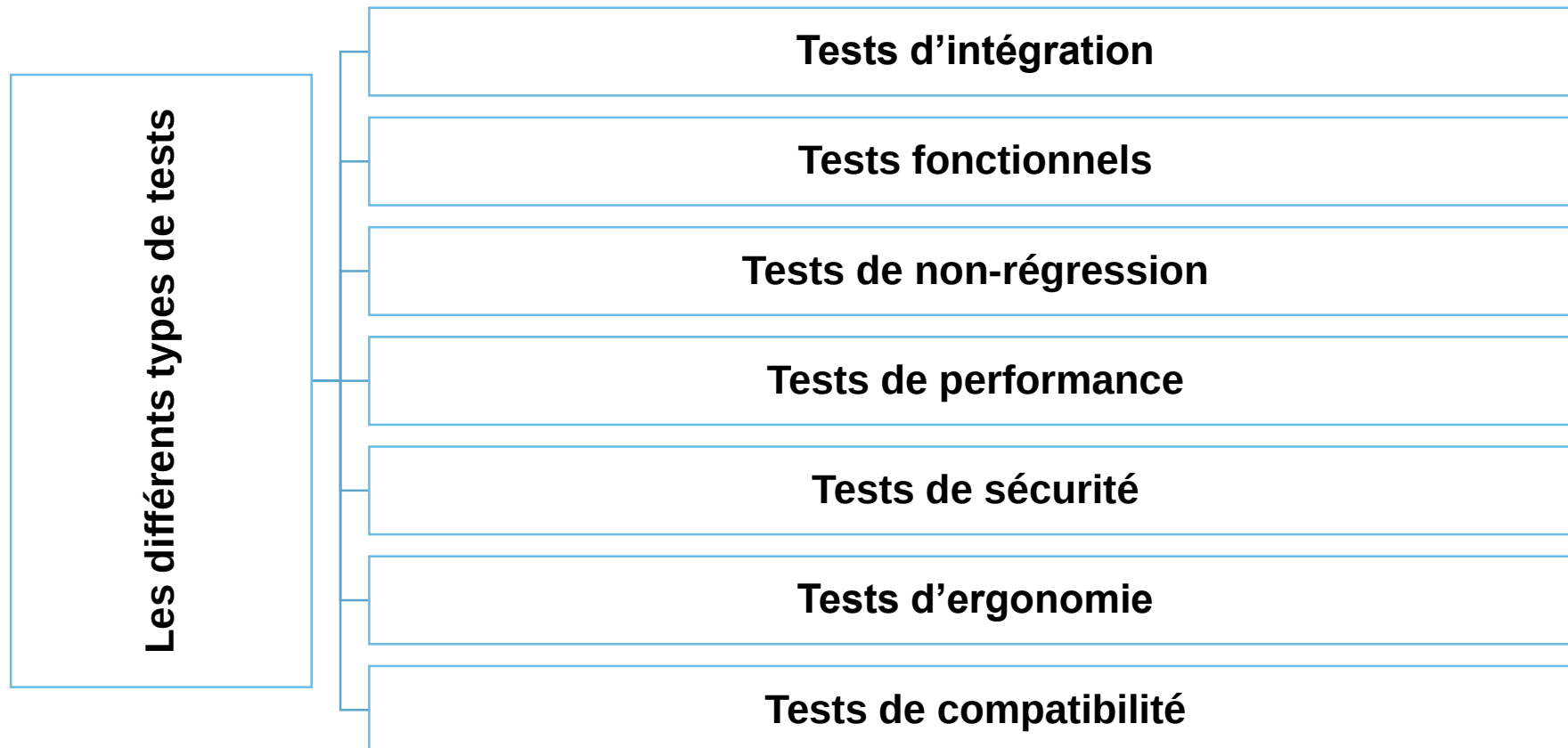


Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.2 Les différents types de tests



Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.2. Les différents types de tests

a) Tests d'intégration

Objectif

Vérifier que les **modules fonctionnent correctement ensemble**

Périmètre

- ❑ Intégration **verticale** : Présentation → Métier → Données
- ❑ Intégration **horizontale** : modules de même niveau
- ❑ Communication avec :
 - Bases de données
 - Services web tiers
 - Systèmes legacy
 - Services d'infrastructure (LDAP, mail...)

Approches

- ❑ **Big Bang** : tout intégrer en une seule fois (simple mais risqué)
- ❑ **Incrémentale** : Top-down, Bottom-up, Par sous-systèmes fonctionnels

b) Tests fonctionnels

Objectif

Vérifier que les **fonctionnalités du cahier des charges** sont respectées

Base des tests

- ❑ Cahier des charges & spécifications
- ❑ Traçabilité : **Exigence** ↔ **Cas de test**

Scénarios

- ❑ Cas nominaux
- ❑ Cas limites
- ❑ Cas d'erreur
- ❑ Chemins alternatifs

Vérifications

- ❑ Règles de gestion métier
- ❑ Conformité fonctionnelle
- ❑ Cohérence des données

Exécution

- ❑ Manuelle ou automatisée
- ❑ Outils : Selenium, Cypress, Katalon, UFT

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.2. Les différents types de tests

c) Tests de non-régression

Objectif

Garantir que les fonctionnalités existantes **continuent de fonctionner**

Principe

- ❑ Rejouer une **suite de tests critiques** après chaque modification
- ❑ Ajouter un test à chaque bug corrigé

Automatisation

- ❑ Fortement recommandée

d) Tests de performance

Objectif

Vérifier le comportement du système **sous charge**

Types

- ❑ **Tests de charge** : charge normale
- ❑ **Tests de stress** : au-delà des limites
- ❑ **Tests d'endurance** : longue durée
- ❑ **Tests de scalabilité** : montée en charge

Métriques

- ❑ Temps de réponse (moyen, P90, P95...)
- ❑ Débit (transactions/sec)
- ❑ Utilisation des ressources
- ❑ Taux d'erreur

Outils : JMeter, Gatling, LoadRunner, k6

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.2. Les différents types de tests

e) Tests de sécurité

Objectif

Vérifier que le système est **résistant aux attaques**

Domaines testés

- ❑ Authentification & autorisation
- ❑ Attaques courantes : SQL Injection, XSS, CSRF, Command injection
- ❑ Chiffrement (en transit / au repos)
- ❑ Conformité réglementaire : RGPD, PCI-DSS (si applicable)

Méthodes

- ❑ Tests de pénétration (pentests)
- ❑ Audits de sécurité
- ❑ Scans automatisés (OWASP ZAP, Nessus...)

f) Tests d'ergonomie et d'utilisabilité

Objectif

Évaluer l'**expérience utilisateur (UX)**

Aspects testés

- ❑ Facilité de prise en main
- ❑ Intuitivité
- ❑ Cohérence graphique
- ❑ Accessibilité (WCAG)

Méthodes

- ❑ Tests utilisateurs
- ❑ Tests d'utilisabilité formels
- ❑ Tests A/B

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.2. Les différents types de tests

g) Tests de compatibilité

Objectif

Garantir le bon fonctionnement dans **différents environnements**

Tests réalisés sur

- ❑ Navigateurs : Chrome, Firefox, Edge, Safari
- ❑ Systèmes : Windows, Linux, macOS
- ❑ Appareils : PC, mobile, tablette
- ❑ Résolutions & orientations

Versions logicielles (OS, langages, DB, navigateurs)

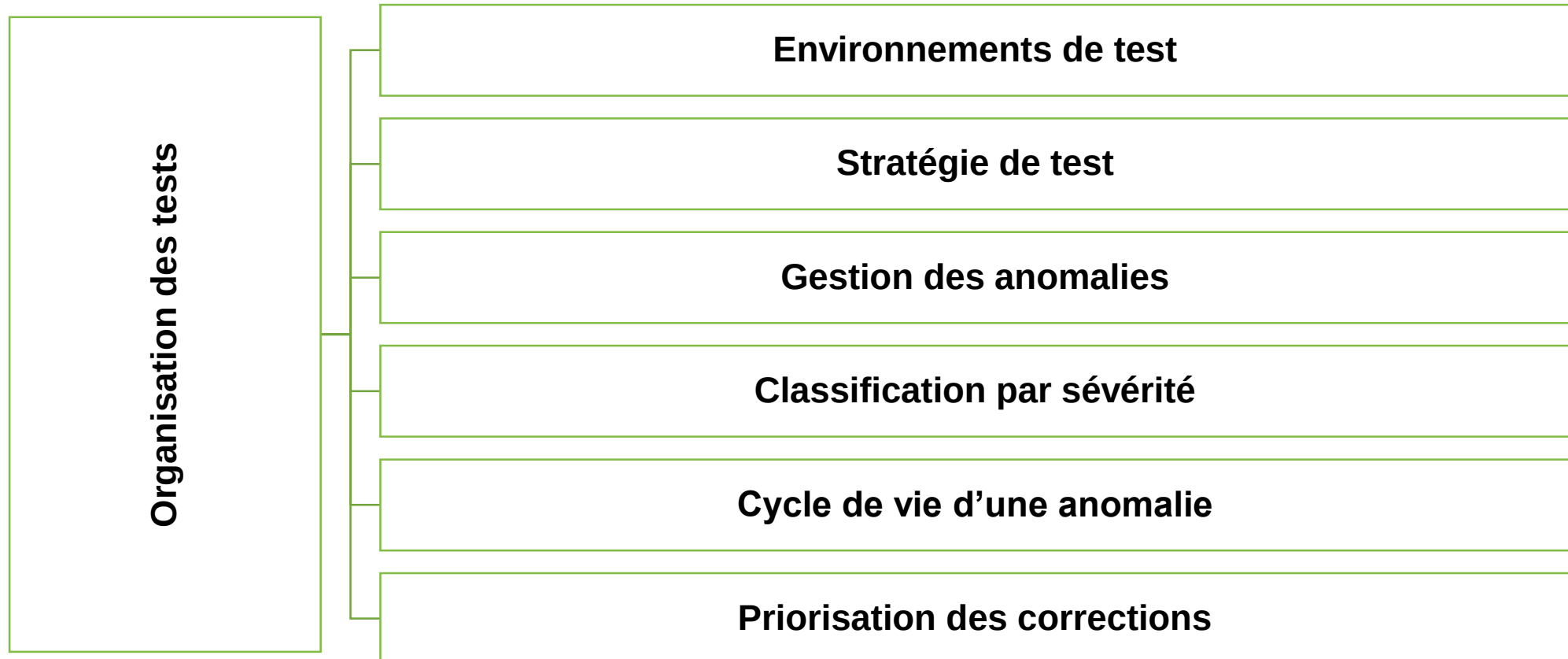
☞ La validation repose sur une **combinaison complémentaire de tests** pour garantir qualité, sécurité, performance et satisfaction utilisateur.

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.3. Organisation des tests



Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.3. Organisation des tests

Environnements de test

On distingue généralement plusieurs environnements avec des rôles spécifiques.

L'environnement de développement (DEV) est utilisé par les développeurs pour les tests unitaires et l'intégration locale. Il est instable et change fréquemment.

L'environnement d'intégration (INT) sert aux tests d'intégration avec un code plus stable que DEV. L'intégration continue y déploie automatiquement.

L'environnement de recette (UAT - User Acceptance Testing) permet les tests fonctionnels et la validation par les utilisateurs. Il est le plus proche de la production en termes de configuration.

L'environnement de pré-production (PREPROD) réplique exactement la production pour les tests finaux avant déploiement. Il permet de tester les procédures de déploiement.

L'environnement de production (PROD) est l'environnement réel où les utilisateurs finaux travaillent.

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.3. Organisation des tests

Stratégie de test

Un **plan de test** définit :

- Les objectifs des tests
- Le périmètre (inclus / exclu)
- Les ressources (équipe, environnements, données, outils)
- Le planning et les jalons
- Les critères de succès (couverture, seuil de bugs critiques)

Les **cas de test** décrivent :

- Les préconditions
- Les étapes à exécuter
- Les données d'entrée
- Les résultats attendus
- Les critères de validation

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.3. Organisation des tests

Gestion des anomalies

Les anomalies sont enregistrées dans des outils de suivi tels que : Jira, Mantis, Redmine ou Bugzilla.

Chaque anomalie comprend :

- Description du problème
- Étapes de reproduction
- Preuves (captures, logs)
- Sévérité et priorité

Classification par sévérité

- **Bloquante** : empêche l'usage ou provoque une perte de données
- **Majeure** : impact important avec solution de contournement
- **Mineure** : gêne limitée
- **Cosmétique** : défaut visuel sans impact fonctionnel

Cycle de vie d'une anomalie

États possibles :

- Nouvelle
- Assignée
- En cours de correction
- Résolue
- Vérifiée
- Fermée
- Rouverte

Priorisation des corrections

- Basée sur la **criticité métier** et l'**urgence**
- Les anomalies **bloquantes** sont traitées en **priorité**, même si cela retarde d'autres développements

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.4. Critères de sortie de la validation

Critères de sortie de la validation

Tests critiques réussis

Anomalies maîtrisées

Performances conformes

Sécurité validée

Documentation complète

Approbation formelle

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.4. Critères de sortie de la validation

La phase de validation est considérée comme terminée lorsque **toutes les conditions suivantes sont réunies** :

✓ Tests critiques réussis

- Toutes les fonctionnalités critiques et les parcours utilisateurs principaux sont validés
- Le taux de couverture minimal des exigences est atteint (ex. $\geq 80\%$)

✓ Anomalies maîtrisées

- Aucune anomalie bloquante en attente
- Les anomalies majeures sont corrigées ou formellement acceptées avec un plan d'action
- Les anomalies mineures acceptées sont documentées et planifiées

✓ Performances conformes

- Les tests confirment le respect des objectifs :
 - Temps de réponse
 - Capacité de charge
 - Stabilité du système

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.4. Critères de sortie de la validation

✓ Sécurité validée

- Aucune vulnérabilité critique détectée
- Respect des exigences de sécurité et de conformité réglementaire (ex. protection des données)

✓ Documentation complète

- Documentation utilisateur finalisée et validée
- Documentation technique à jour
- Procédures d'exploitation prêtes

✓ Approbation formelle

- Validation par le responsable qualité ou l'équipe de test
- Accord du maître d'ouvrage pour passer à la phase de recette

Chapitre 3 : Les étapes projet

3.3. La Validation

3.3. La Validation

3.3.5. Livrables de la validation

Les principaux livrables de cette phase sont :

- **Plan de test et scénarios de test détaillés**
- **Rapports d'exécution des tests** (succès / échecs)
- **Indicateurs de couverture des tests**
- **Registre des anomalies** avec leur statut (corrigée, acceptée, reportée)
- **Procès-verbaux de validation** signés par les parties prenantes
- **Dossier de validation** consolidé concluant sur l'aptitude du système à passer en recette

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.1. Définition et distinction

La **recette** est la phase durant laquelle le **maître d'ouvrage** (ou ses représentants) vérifie que le système livré correspond bien à **ses attentes** et au cahier des charges initial. C'est une validation **métier** effectuée dans un environnement dédié (environnement de recette) qui reproduit au mieux les conditions réelles d'utilisation.

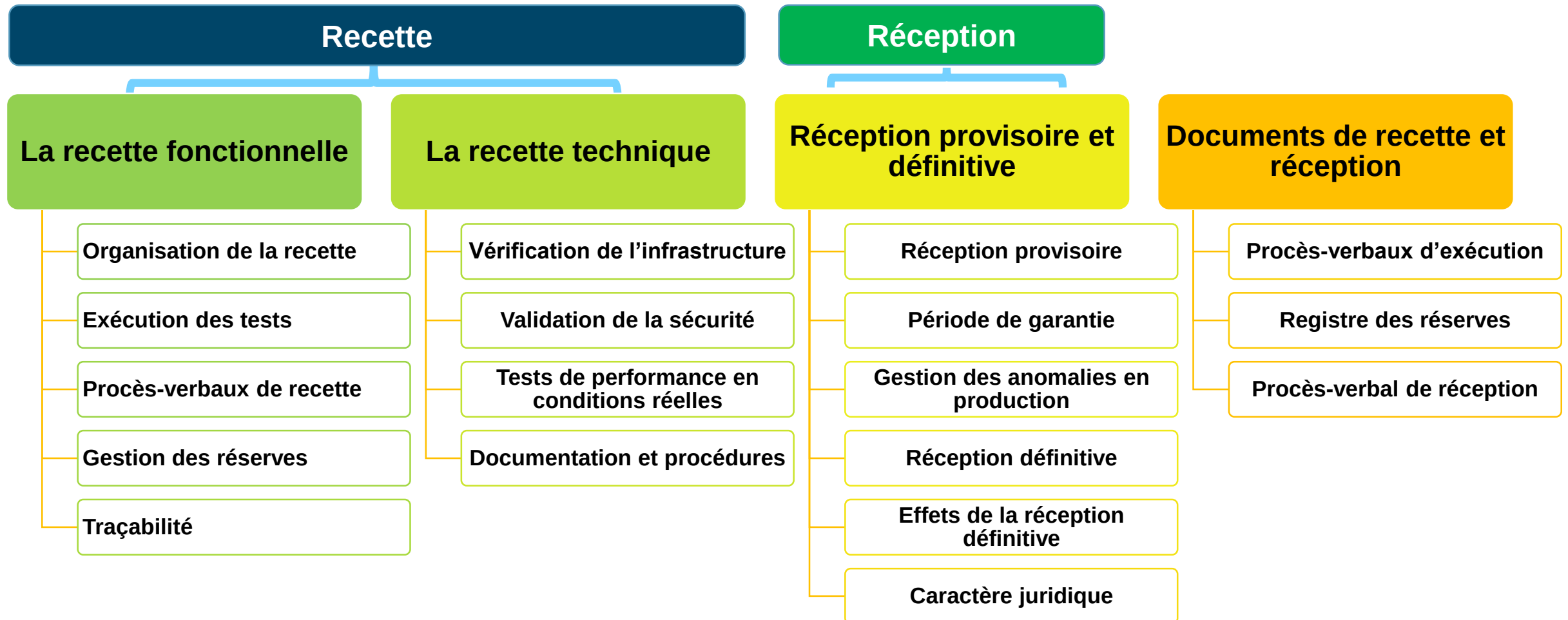
La **réception** est **l'acte formel** par lequel le maître d'ouvrage accepte officiellement le système et autorise sa mise en production. Elle marque le **transfert de responsabilité** du maître d'œuvre (qui a développé) vers le maître d'ouvrage (qui va exploiter). C'est un jalon contractuel majeur.

La distinction est importante : la recette est un **processus** (ensemble d'activités de vérification), tandis que la réception est un **événement** (décision formalisée par un procès-verbal).

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception



Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.2. La recette fonctionnelle

Objectifs

La recette fonctionnelle vise à :

- Vérifier que **toutes les fonctionnalités du cahier des charges** sont présentes et opérationnelles
- Valider l'adéquation du système aux **besoins métier réels**
- Confirmer que les **processus de l'organisation** sont correctement supportés
- Évaluer l'**ergonomie** et la facilité d'utilisation pour les utilisateurs finaux

Organisation de la recette

Le **maître d'ouvrage (MOA)** :

- Prépare les scénarios de recette à partir du cahier des charges et des processus métier
- Sélectionne et mobilise les **utilisateurs clés (key users)** représentant les différents profils
- Fournit des **jeux de données réalistes** (volume, variété, complexité proches du réel)

Exécution des tests

- Tests réalisés par les **utilisateurs finaux** ou leurs représentants
- Conditions proches du **fonctionnement réel**
- Utilisation de **données métier significatives**
- Chaque scénario précise :
 - Les étapes à exécuter
 - Les résultats attendus

Procès-verbaux de recette

Chaque test fait l'objet d'un procès-verbal avec un statut :

- **Succès** : résultat conforme aux attentes
- **Échec** : anomalie détectée, non-conformité
- **Réserves** : test globalement conforme avec écarts mineurs

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.2. La recette fonctionnelle

Gestion des réserves

Les écarts identifiés sont classifiés :

Réserves majeures (bloquantes)	Réserves mineures	Demandes d'évolution
▪ Empêchent l'utilisation du système pour des processus critiques ▪ Bloquent la réception ▪ Correction obligatoire avant acceptation	▪ Gênantes mais non bloquantes ▪ Contournements possibles ▪ N'empêchent pas la réception ▪ Correction planifiée dans un délai convenu	▪ Améliorations hors périmètre initial ▪ Non bloquantes ▪ Intégrées au backlog de maintenance évolutive

Traçabilité

- Chaque exigence du cahier des charges doit être liée à un ou plusieurs tests de recette
- Une **matrice de traçabilité** permet de :
 - Vérifier que toutes les exigences ont été testées
 - Garantir l'exhaustivité de la recette

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.3. La recette technique

Définition

- Menée **en parallèle de la recette fonctionnelle**
- Réalisée par :
 - Équipes d'**exploitation**
 - **Architectes techniques**
- Objectif : valider que le système est **techniquement prêt pour la production**

Validation de la sécurité

- **Contrôles d'accès**
 - Authentification fonctionnelle
 - Autorisations correctes
 - Ségrégation des données entre utilisateurs
- **Chiffrement**
 - HTTPS actif (certificats valides)
 - Données sensibles chiffrées
 - Mots de passe stockés de façon sécurisée (hash + salt)
- **Traçabilité & conformité**
 - Logs d'accès et d'actions critiques
 - Respect des exigences **RGPD**

Tests de performance en conditions réelles

- Tests avec :
 - Volumes de données réels ou représentatifs
 - Nombre réel d'utilisateurs simultanés
- Mesure des **temps de réponse réels**
- Validation de la stabilité sous charge

Vérification de l'infrastructure

- **Serveurs**
 - Dimensionnement adéquat
 - Configuration correcte
 - Sécurisation validée
- **Réseau**
 - Bande passante suffisante
 - Segmentation sécurisée
- **Sauvegardes**
 - Sauvegardes opérationnelles
 - Procédures de **restauration testées**
- **Supervision**
 - Outils de monitoring actifs
 - Alertes correctement configurées

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.3. La recette technique

Documentation et procédures

- Vérification de la documentation :
 - ✓ Architecture technique
 - ✓ Procédures d'exploitation
 - ✓ Guides de dépannage
 - ✓ Contacts d'escalade
- Validation des **procédures de déploiement**
 - ✓ Tests réels en pré-production
- Test des **procédures de rollback**
 - ✓ Capacité de retour arrière en cas d'incident

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.4. Réception provisoire et définitive

Réception provisoire (recette avec réserves)

Principe

- Prononcée si :
 - Réserves **mineures**
 - **Aucun problème bloquant**
- Autorise la **mise en production**
- Réserves à lever dans un **délai convenu**

Période de garantie

- Entre réception provisoire et définitive
- Responsabilités du **maître d'œuvre** :
 - Correction des anomalies résiduelles
 - Support renforcé
 - Ajustements mineurs (sans coût supplémentaire)
- Rôle du **maître d'ouvrage** :
 - Suivi de la levée des réserves
 - Validation des corrections

Gestion des anomalies en production

- Les nouvelles anomalies :
 - Traitées dans le cadre de la **garantie**
- Validation progressive jusqu'à levée complète des réserves

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.4. Réception provisoire et définitive

Réception définitive

Conditions

- ☐ Toutes les réserves **levées**
- ☐ Période d'exploitation **satisfaisante**

Effets de la réception définitive

- ☐ **Clôture officielle du projet**
- ☐ Passage du **mode projet** au **mode maintenance**
- ☐ **Païement final** du maître d'œuvre (si sous-traitance)
- ☐ **Transfert complet de responsabilité** au maître d'ouvrage

Caractère juridique

- ☐ Réception généralement **irrévocable**
- ☐ Plus de réclamation possible :
 - Sauf cas exceptionnels de **vices cachés graves**

☞ La réception provisoire sécurise la mise en production, la réception définitive marque la fin du projet et le début de l'exploitation durable.

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.5. Documents de recette et réception

Plan de recette

- ❑ Décrit le **déroulement global de la recette**
- ❑ Contenu :
 - Scénarios de test à exécuter
 - **Planning** (dates, durée)
 - **Responsabilités** (qui teste quoi)
 - Jeux de données
 - **Critères d'acceptation**
- ❑ Sert de **référence officielle** pour la recette

Fiches de test de recette

- ❑ Une fiche par **scénario de test**
- ❑ Contenu :
 - Identifiant unique
 - Objectif du test
 - Préconditions
 - Étapes détaillées
 - Résultats attendus
 - Résultats réels & observations
- ❑ Assurent la **traçabilité** des tests

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.5. Documents de recette et réception

Procès-verbaux d'exécution

- ☐ Rédigés après chaque test exécuté
- ☐ Contiennent :
 - Date d'exécution
 - Testeur
 - Résultat :
 - Succès
 - Échec
 - Réserve
 - Anomalies détectées
 - Signatures des parties
- ☐ Preuve formelle de l'**exécution des tests**

☐ Registre des réserves

- ☐ Centralise toutes les **réserves émises**
- ☐ Pour chaque réserve :
 - Description
 - Classification (majeure / mineure)
 - Plan de levée (qui, quand, comment)
 - Statut (en cours / levée / acceptée)
- ☐ Outil clé de **suivi post-recette**

Chapitre 3 : Les étapes projet

3.4. La Recette et la Réception

3.4. La Recette et la Réception

3.4.5. Documents de recette et réception

Procès-verbal de réception

- Document **contractuel officiel**
- Signé par le **maître d'ouvrage**
- Contenu :
 - Conformité globale du système
 - Liste des réserves éventuelles
 - Type de réception : Provisoire, Définitive
 - Délais de levée des réserves
- Déclenche : Paiements, Garantie, Responsabilités contractuelles

☞ Les documents de recette sécurisent juridiquement et opérationnellement la mise en production et la réception du projet.

Chapitre 3 : Les étapes projet

3.5. La Livraison

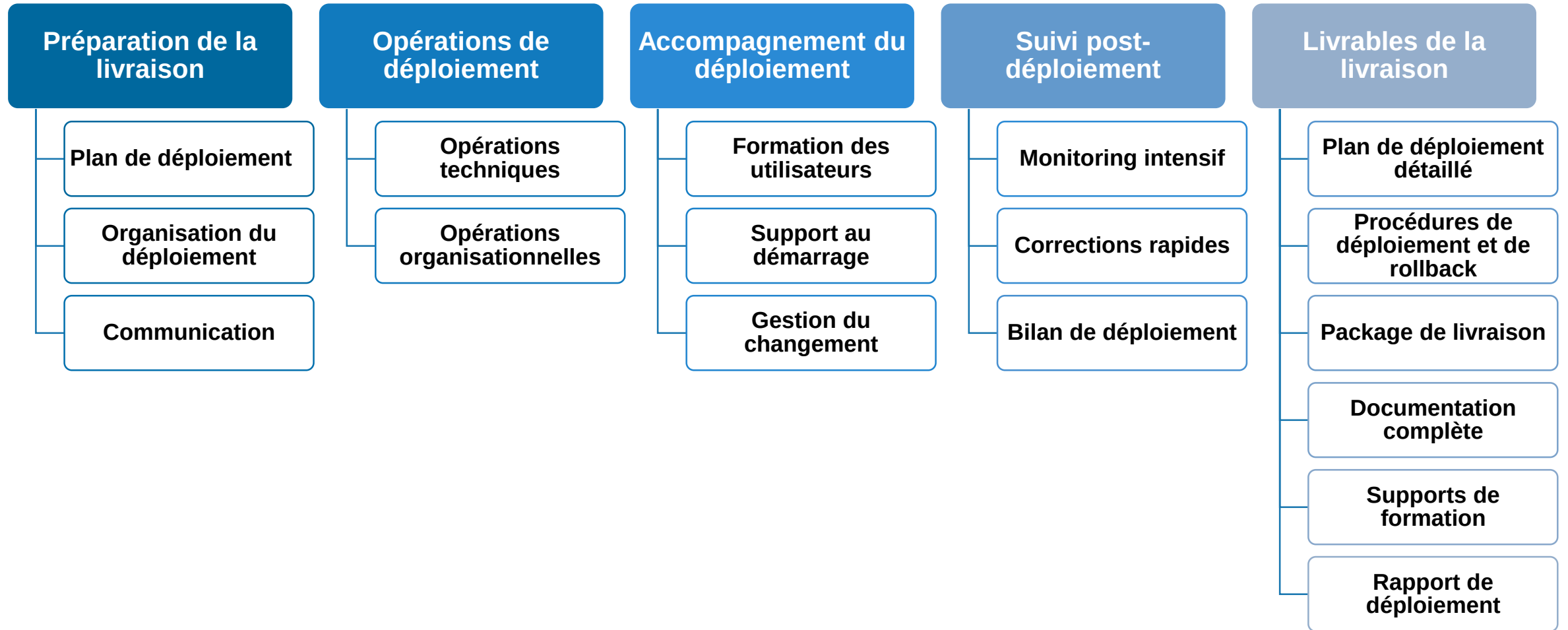
3.5. La Livraison

La livraison correspond au **passage effectif du système vers la production** et à son appropriation par les utilisateurs. Elle ne se limite pas à **un acte technique** : elle inclut **organisation, communication, formation et accompagnement.**

Chapitre 3 : Les étapes projet

3.5. La Livraison

3.5. La Livraison



Chapitre 3 : Les étapes projet

3.5. La Livraison

3.5. La Livraison

3.5.1. Préparation de la livraison

Finalisation des livrables

Avant toute mise en production, tous les éléments doivent être **complets, validés et à jour** :

- ❑ **Code** finalisé, documenté et versionné (release)
- ❑ **Exécutables** compilés, packagés et testés
- ❑ **Documentation complète** :
 - Utilisateur (manuels, guides rapides, FAQ)
 - Technique (architecture, API, code)
 - Exploitation (installation, configuration, supervision, dépannage)
- ❑ **Scripts prêts et testés** :
 - Installation et configuration
 - Migration de bases de données
 - Rollback (retour arrière)
 - Vérifications post-déploiement
- ❑ **Données préparées** :
 - Paramétrage
 - Données de test
 - Données migrées de l'ancien système

Chapitre 3 : Les étapes projet

3.5. La Livraison

3.5. La Livraison

3.5.1. Préparation de la livraison

Plan de déploiement

Document central qui organise la mise en production.

Stratégies de déploiement

- ❑ **Big bang** : déploiement global en une seule fois
✓ rapide | ✗ risqué
- ❑ **Déploiement progressif (phased rollout)** : par site, service, profil ou fonctionnalité
✓ maîtrise des risques | ✗ transition plus longue
- ❑ **Déploiement pilote** : groupe restreint d'utilisateurs
✓ validation terrain | ✗ ressources supplémentaires
- ❑ **Déploiement parallèle** : ancien et nouveau système en coexistence
✓ très sécurisé | ✗ coûteux et lourd

Organisation du déploiement

- **Planning détaillé** : dates, horaires, durées, fenêtres de maintenance, points go/no-go
- **Rôles et responsabilités** :
 - Chef de projet : coordination et décisions
 - Équipe technique : déploiement
 - Exploitation : infrastructure et supervision
 - Support : assistance utilisateurs
 - Key users : validation et relais terrain
- **Prérequis** : infrastructure prête, licences, comptes, sauvegardes
- **Plan de rollback** : conditions, procédures, point de non-retour
- **Critères de succès** :
 - Système accessible
 - Fonctions critiques opérationnelles
 - Performances conformes
 - Aucun bug bloquant
 - Adoption conforme aux prévisions

Communication

- **Vers les utilisateurs** : planning, impacts, interruptions, contacts support
- **Vers le support** : formation, FAQ, anticipation des incidents
- **Vers la direction** : avancement, risques, arbitrages

Chapitre 3 : Les étapes projet

3.5. La Livraison

3.5. La Livraison

3.5.2. Opérations de déploiement

Opérations techniques

- Installation / mise à jour serveurs et applications
- **Migration des données** (ETL) : extraction, transformation, chargement, contrôle
- Configuration production :
 - Connexions, paramètres, sécurité, supervision
- Activation :
 - Services, batchs, accès utilisateurs
- Vérifications :
 - Smoke tests
 - Connexions externes
 - Performances initiales
- Sauvegarde complète pour rollback

Opérations organisationnelles

- ☐ Bascule des utilisateurs vers le nouveau système
- ☐ Activation du support renforcé
- ☐ Monitoring technique et suivi de l'adoption

Chapitre 3 : Les étapes projet

3.5. La Livraison

3.5. La Livraison

3.5.3. Accompagnement du déploiement

Formation des utilisateurs

☐ Formes :

- Présentiel
- Distanciel (webinaires)
- E-learning et vidéos
- Guides, cheat sheets, ateliers pratiques

☐ Adaptation aux profils :

- Utilisateurs finaux
- Power users
- Administrateurs
- Support

☐ Timing :

- Juste avant le déploiement
- Rappels après quelques semaines

Gestion du changement

- ☐ Communication orientée **bénéfices**
- ☐ Implication des utilisateurs en amont
- ☐ Identification des champions du changement
- ☐ Traitement des résistances
- ☐ Adaptation des processus métier

Support au démarrage

- ☐ Assistance sur site
- ☐ Hotline dédiée
- ☐ Chat / messagerie
- ☐ FAQ évolutive
- ☐ **Key users** comme relais de proximité

Chapitre 3 : Les étapes projet

3.5. La Livraison

3.5. La Livraison

3.5.4. Suivi post-déploiement

Monitoring intensif

- **Performances** : temps de réponse, charge, disponibilité
- **Erreurs** : logs, exceptions, récurrences
- **Utilisation** : adoption, fonctionnalités utilisées
- **Incidents** : volume, délais de résolution, satisfaction

Corrections rapides

- Bugs critiques → hot fixes prioritaires
- Ajustements de configuration
- Améliorations ergonomiques rapides

Bilan de déploiement

- Respect du **planning et du budget**
- Analyse des incidents
- Satisfaction des utilisateurs
- Efficacité des formations et du support
- Ajustements pour la suite

Chapitre 3 : Les étapes projet

3.5. La Livraison

3.5. La Livraison

3.5.5. Livrables de la livraison

- Plan de déploiement détaillé
- Procédures de déploiement et de rollback
- Package de livraison (code, exécutables, configs, données)
- Documentation complète
- Supports de formation
- Rapport de déploiement (déroulement, incidents, ajustements)

☞ *La livraison est une phase critique où la réussite dépend autant de la **technique** que de la **communication, la formation et l'accompagnement humain**.*

Chapitre 3 : Les étapes projet

3.6. Le Bilan de Fin de Projet

3.6. Le Bilan de Fin de Projet

Le bilan de fin de projet permet de **capitaliser sur l'expérience**, évaluer la réussite du projet, formaliser la clôture administrative et préparer la transition vers l'exploitation et la maintenance.

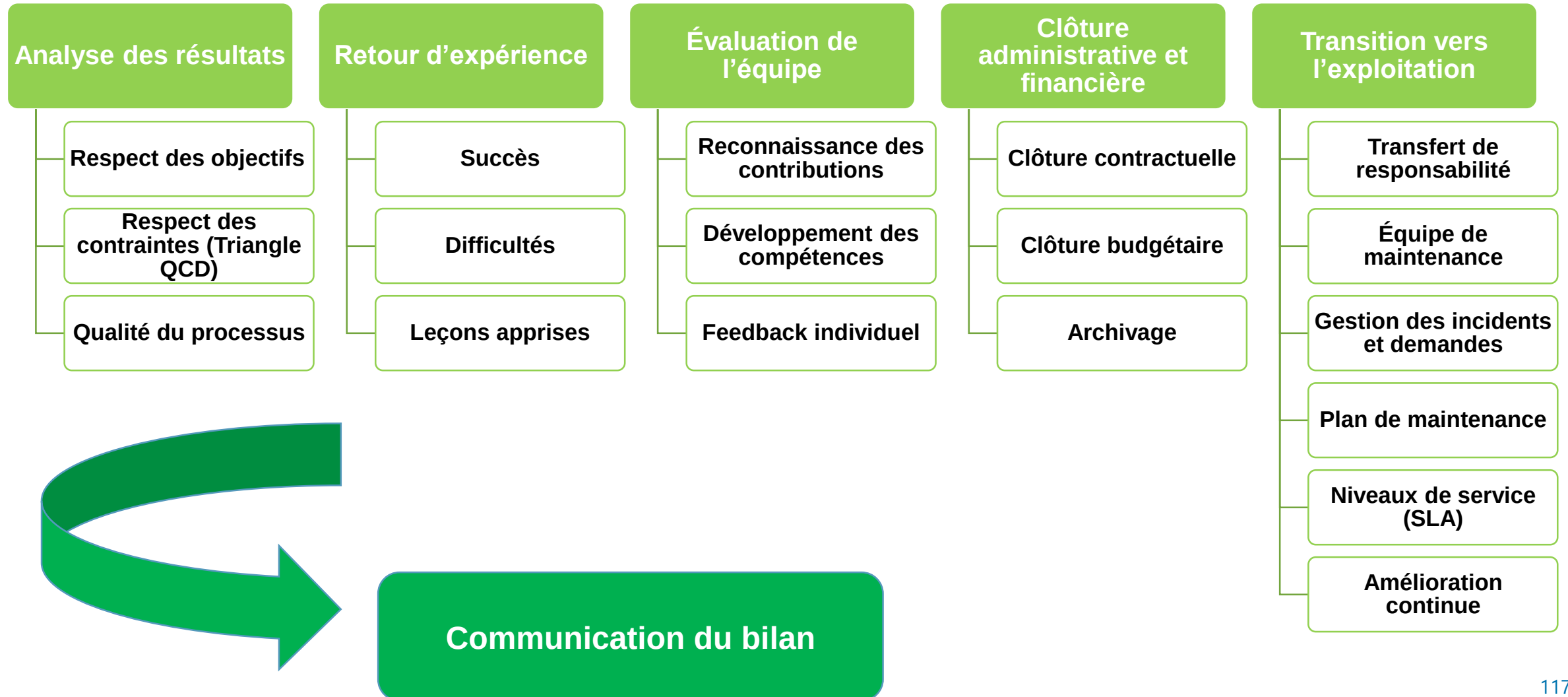
3.6.1. Objectifs du bilan

- Évaluer la réussite par rapport aux objectifs initiaux
- Identifier bonnes pratiques et erreurs à éviter
- Clôturer le projet sur les plans administratif et financier
- Préparer la transition vers l'exploitation et la maintenance
- Favoriser l'apprentissage organisationnel et l'amélioration continue

Chapitre 3 : Les étapes projet

3.6. Le Bilan de Fin de Projet

3.6. Le Bilan de Fin de Projet



Chapitre 3 : Les étapes projet

3.6. Le Bilan de Fin de Projet

3.6. Le Bilan de Fin de Projet

3.6.2. Analyse des résultats

Respect des objectifs

- Atteinte des objectifs initiaux
- Comparaison fonctionnalités livrées vs périmètre initial
- Qualité du livrable : performances, fiabilité, ergonomie, sécurité
- Bénéfices réels : gains de productivité, réduction des coûts, satisfaction client
- Adoption : taux d'utilisation, appropriation des nouvelles pratiques

Respect des contraintes (Triangle QCD)

- **Délais** : comparatif planning réel vs prévu, causes et impact des retards
- **Budget** : analyse des écarts, coûts par poste, ROI réel
- **Périmètre** : dérives éventuelles (scope creep), justification et impact

Qualité du processus

- Méthodologie : pertinence et application (Agile, Cycle en V, hybride)
- Planification et pilotage : suivi, décisions, réactivité
- Communication et collaboration : efficacité, gestion des conflits
- Outils et techniques : pertinence et adoption

Chapitre 3 : Les étapes projet

3.6. Le Bilan de Fin de Projet

3.6. Le Bilan de Fin de Projet

3.6.3. Retour d'expérience

Succès

- Bonnes pratiques, facteurs de succès techniques, méthodologiques ou organisationnels
- Décisions clés judicieuses et moments de collaboration réussis
- Compétences ou expertises qui ont fait la différence

Difficultés

- **Techniques** : bugs tardifs, limites des technologies, intégration
- **Organisationnelles** : communication, coordination, processus lourds
- **Humaines** : compétences manquantes, surcharge, turnover
- **Externes** : fournisseurs, régulations, contraintes budgétaires

Leçons apprises

- Points de vigilance pour projets futurs
- Recommandations méthodologiques
- Besoins de formation
- Améliorations des processus et outils

Chapitre 3 : Les étapes projet

3.6. Le Bilan de Fin de Projet

3.6. Le Bilan de Fin de Projet

3.6.4. Évaluation de l'équipe

- **Reconnaissance des contributions** : performances, initiatives, esprit d'équipe
- **Développement des compétences** : techniques, méthodologiques, transverses
- **Feedback individuel** : satisfaction, difficultés, enseignements et souhaits

3.6.5. Clôture administrative et financière

- **Clôture contractuelle** : levée des réserves, paiements, garanties
- **Clôture budgétaire** : analyse finale, comparaison budget initial vs réel, justification des écarts
- **Archivage** : cahier des charges, livrables techniques, documents contractuels, documentation opérationnelle

Chapitre 3 : Les étapes projet

3.6. Le Bilan de Fin de Projet

3.6. Le Bilan de Fin de Projet

3.6.6. Transition vers l'exploitation

- **Transfert de responsabilité** vers l'équipe de maintenance
- **Équipe de maintenance** : formation, partage des points critiques et bugs connus
- **Gestion des incidents et demandes** : processus, classification, SLA, escalades
- **Plan de maintenance** :
 - Corrective : bugs production
 - Évolutive : nouvelles fonctionnalités
 - Adaptative : évolutions réglementaires et environnementales
 - Préventive : audits, optimisation, veille technologique
- **Niveaux de service (SLA)** : délais, disponibilité, résolution, plages de support
- **Amélioration continue** : quick wins, optimisations, roadmap produit

Chapitre 3 : Les étapes projet

3.6. Le Bilan de Fin de Projet

3.6. Le Bilan de Fin de Projet

3.6.7. Communication du bilan

- **Réunion de clôture** : présentation des résultats, enseignements, perspectives, remerciements
- **Rapport de bilan** : synthèse des résultats, écarts, leçons, perspectives de maintenance
- **Partage des connaissances** :
 - Présentations internes pour équipes projet et métier
 - Mise à jour des référentiels méthodologiques
 - Contributions à la communauté interne ou externe (blogs, conférences)